

TIC-VLA: A Think-in-Control Vision-Language-Action Model for Robot Navigation in Dynamic Environments

Zhiyu Huang^{*1} Yun Zhang^{*1} Johnson Liu¹ Rui Song¹ Chen Tang¹ Jiaqi Ma¹

Abstract

Robots in dynamic, human-centric environments must follow language instructions while maintaining real-time reactive control. Vision-language-action (VLA) models offer a promising framework, but they assume temporally aligned reasoning and control, despite semantic inference being inherently delayed relative to real-time action. We introduce **Think-in-Control (TIC)-VLA**, a latency-aware framework that explicitly models delayed semantic reasoning during action generation. TIC-VLA defines a delayed semantic-control interface that conditions action generation on delayed vision-language semantic states and explicit latency metadata, in addition to current observations, enabling policies to compensate for asynchronous reasoning. We further propose a latency-consistent training pipeline that injects reasoning inference delays during imitation learning and online reinforcement learning, aligning training with asynchronous deployment. To support realistic evaluation, we present **DynaNav**, a physics-accurate, photo-realistic simulation suite for language-guided navigation in dynamic environments. Extensive experiments in simulation and on a real robot show that TIC-VLA consistently outperforms prior VLA models while maintaining robust real-time control under multi-second reasoning latency. Project website: <https://ucla-mobility.github.io/TIC-VLA/>

1. Introduction

Robots operating in real-world, human-centric environments must react to dynamic scenes while following high-level natural language instructions (Chen et al., 2025b). Vision-

^{*}Equal contribution ¹University of California, Los Angeles. Correspondence to: Zhiyu Huang <zhiyuh@ucla.edu>, Yun Zhang <yun666@ucla.edu>, Jiaqi Ma <jiaqima@ucla.edu>.

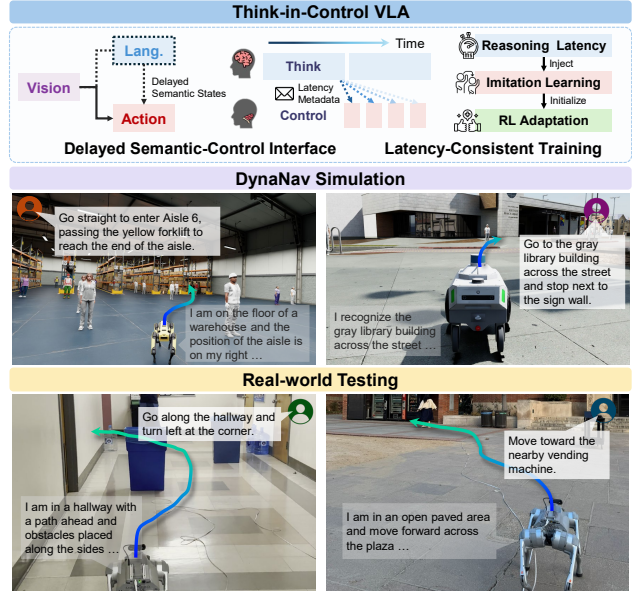


Figure 1. TIC-VLA enables real-time, language-conditioned navigation by decoupling slow vision-language reasoning from fast reactive control via a delayed semantic-control interface. A latency-consistent training strategy improves robustness under variable reasoning delays. Performance is demonstrated in the DynaNav simulation and real-world indoor and outdoor navigation tasks.

language-action (VLA) models (Hirose et al., 2025; Xu et al., 2024; Driess et al., 2026) have emerged as a promising paradigm by unifying perception, language understanding, and control within a single learning-based system. By incorporating large vision-language models (VLMs), these approaches enable semantic grounding, task reasoning, and instruction following (Zhou et al., 2026; Choi et al., 2024; Zhao et al., 2025; Lin et al., 2025; Chandaka et al., 2025; Castro et al., 2025). However, existing VLA systems rely on a hidden and impractical assumption: reasoning and control are temporally aligned. Vision-language reasoning is produced intermittently and often with substantial delay, while control must operate continuously as the robot moves and the environment changes. As a result, semantic representations frequently correspond to past world states, yet are consumed by the policy as if they were current, introducing systematic misalignment between thinking and control. This temporal mismatch is particularly pronounced on mobile

robots with limited computation, where VLM inference can take seconds while control loops run at tens of hertz.

Most prior work on embodied navigation sidesteps this issue. Classical vision-language navigation (VLN) abstracts navigation into discrete viewpoint transitions, ignoring embodiment, dynamics, and timing (Krantz et al., 2020; Yu et al., 2026; Raychaudhuri & Chang, 2025; Zeng et al., 2025; Eftekhari et al., 2024; Hu et al., 2025a). Recent VLA systems rely on powerful computers and often pause execution during reasoning inference (Zhang et al., 2024b; Cheng et al., 2024), which is impractical in dynamic environments. Dual-system and asynchronous VLA architectures decouple high-level reasoning from low-level control (Wei et al., 2025a; InternRobotics, 2025), but still assume that semantic outputs are temporally fresh, implicitly treating inference latency as negligible. We argue that latency in reasoning is not merely an engineering inefficiency but a fundamental modeling problem. When semantic information is delayed, it may describe a past state of the robot and the environment, rather than the current one. If this delay is not represented or considered during policy learning, policies trained under idealized synchronous supervision can degrade severely when deployed under realistic inference latency.

To this end, we introduce **Think-in-Control (TIC)-VLA**, a latency-aware framework that explicitly exposes inference delay to the control policy. Rather than enforcing real-time constraints on semantic reasoning, TIC-VLA defines a *delayed semantic-control interface* that allows reasoning to proceed asynchronously while enabling robust real-time control. Specifically, the reasoning module produces delayed latent semantic representations together with explicit latency and ego-motion offset metadata, while the action policy conditions on this delayed semantic-control interface. This interface is the key distinction from prior asynchronous or dual-system designs: instead of only separating slow reasoning and fast control, TIC-VLA makes the temporal staleness of the semantic state an explicit input to the controller. Importantly, architectural decoupling alone is insufficient. Policies trained under idealized synchronous supervision fail when deployed with delayed semantic inputs. We therefore propose a *latency-consistent training pipeline* in which inference delays are explicitly injected during imitation learning and reinforcement learning. This training scheme aligns the policy’s training-time inputs with the delayed semantic information it receives at deployment, enabling robustness to variable reasoning latency rather than assuming synchronous semantic supervision.

To support realistic and reproducible evaluation, we develop **DynaNav**, a simulation suite featuring realistic rendering, dynamic human agents, physics-based execution, and diverse indoor and outdoor scenarios. DynaNav supports teleoperated data collection, online RL, and benchmarking.

Experiments in both simulation and real-world deployment demonstrate that our proposed TIC-VLA model achieves robust navigation under realistic inference latency. Figure 1 illustrates the key design of TIC-VLA and its performance in simulation and real-world environments. The primary contributions can be summarized as:

1. We introduce TIC-VLA with a delayed semantic-control interface that provides the real-time action policy with delayed semantic features, latency metadata, and ego-motion offsets.
2. We propose a latency-consistent training pipeline that aligns learning with asynchronous inference at deployment, yielding robust navigation under variable delays.
3. We present DynaNav, a realistic simulation suite and benchmark for language-guided navigation in dynamic environments, and we demonstrate TIC-VLA’s strong performance in simulation and in real-world settings.

2. Related Work

Learning-based Visual Navigation. Recent advances in robot navigation have shifted from traditional map-based pipelines toward end-to-end learning-based models. Diffusion policies (Cai et al., 2025; Hu et al., 2025b), imitation and reinforcement learning methods (Liu et al., 2025b; Wu et al., 2023; Tang et al., 2025; He et al., 2025), and world modeling approaches (Liu et al., 2025a; Bar et al., 2025) have demonstrated strong performance in navigating complex environments without relying on maps. The integration of LLMs and VLMs into navigation tasks (Yuan et al., 2025; Xu et al., 2024; Zhang et al., 2025b; Gao et al., 2025; Zhou et al., 2024) has further expanded robots’ semantic understanding and open-vocabulary reasoning capabilities, allowing them to follow flexible natural language commands. However, in most of these works, VLMs are employed as auxiliary modules rather than being fully integrated into the navigation pipeline. This limitation has motivated the development of VLA models, which unify perception, instruction, reasoning, and planning within a single framework.

VLA for Navigation. Recent studies increasingly employ VLA models for robotic navigation (Du et al., 2025). Representative methods span direct action prediction and intermediate planning: NaVid (Zhang et al., 2024b) predicts next-step action from monocular RGB inputs, while NaVILA (Cheng et al., 2024) generates mid-level linguistic actions executed by a visual locomotion policy. TrackVLA (Wang et al., 2025b) combines language-based recognition with diffusion-based trajectory planning. Several works further explore generalist VLA frameworks: OmniVLA (Hirose et al., 2025) supports multiple forms of instruction conditioning, including egocentric poses, images, and natural language, while NavFoM (Zhang et al., 2025a) demonstrates strong cross-embodiment performance. More recent

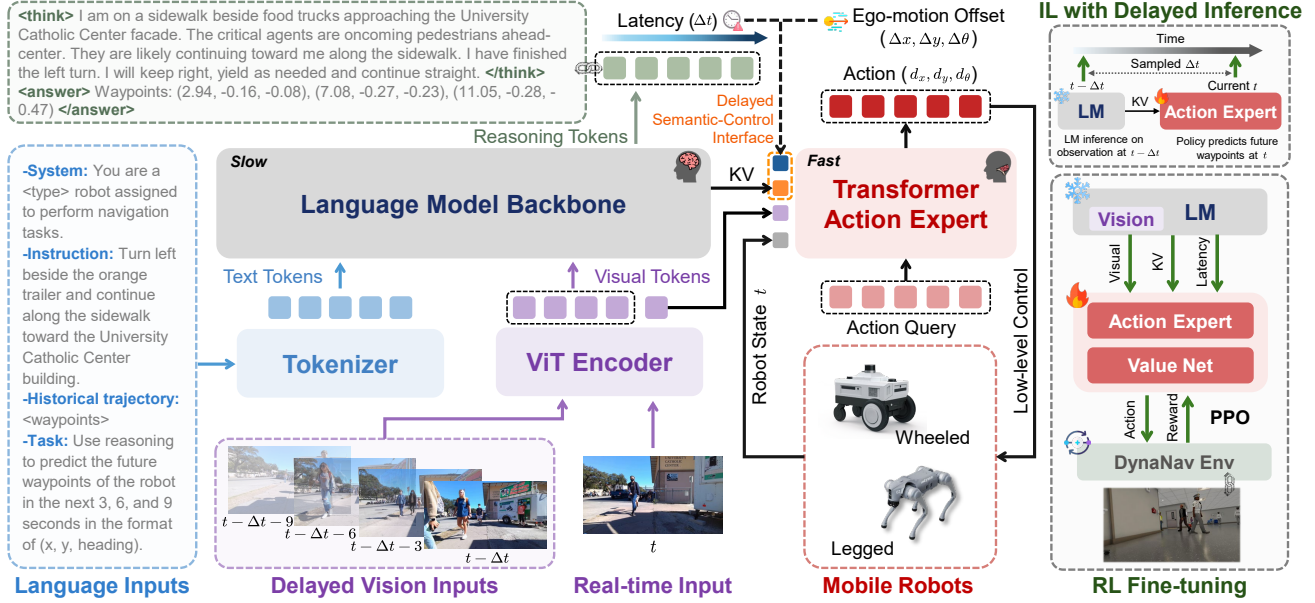


Figure 2. Overview of TIC-VLA. The architecture adopts a decoupled dual-system design with a fast action expert and a slow reasoning VLM. A shared vision encoder provides real-time observations to the policy and time-lagged observations to the VLM, where the delay arises naturally from slow inference. The delayed semantic-control interface (including delayed VLM KV cache features and latency metadata) is explicitly recorded. The Transformer-based action expert takes as input the current observation, robot state, and delayed semantic-control interface data to generate actions from learnable action queries via cross-attention. Multi-stage latency-consistent training combines imitation learning with delayed inference and reinforcement learning to ensure robustness to realistic conditions.

systems address temporal reasoning and real-time execution, such as StreamVLN (Wei et al., 2025b), MobileVLA (Huang et al., 2025), and dual-system VLA approaches like DualVLN (Wei et al., 2025a), which balance deliberative reasoning and reactive control via asynchronous inference. Despite these advances, most existing VLA-based navigation models implicitly assume negligible inference latency, often relying on powerful GPUs or blocking execution during reasoning inference. In contrast, TIC-VLA is designed for latency-aware execution and on-device deployment.

Navigation in Dynamic Environments. Another line of work focuses on navigation in social and dynamic settings. Social-LLaVA (Payandeh et al., 2025b) fine-tunes VLMs for social navigation, while Narrate2Nav (Payandeh et al., 2025a) incorporates implicit language reasoning, social cues, and human intent into visual representations. ViLAD (Elnoor et al., 2026) distills socially compliant navigation knowledge from large VLMs into lightweight policies for real-time deployment. However, these methods typically rely on specialist policies at inference time and do not keep an in-the-loop VLM during execution. Evaluation is further limited by existing simulators. Arena (Shcherbyna et al., 2025) and UrbanSim (Wu et al., 2025) support dynamic agents but lack language-conditioned navigation. SocialNav-SUB (Munje et al., 2025) evaluates real-world social navigation without a controllable simulator, while HA-VLN (Dong et al., 2025) relies on simplified observations and abstracted control. In contrast, *DynaNav* provides

a physics-accurate simulation suite for language-guided navigation in human-centric environments.

3. Method

3.1. Problem Formulation

We consider an instruction-following navigation problem in which an embodied agent must follow natural language instructions to navigate dynamic environments. At each control timestep t , the agent receives: (1) a natural language instruction and context $\mathcal{I}$, specifying the navigation goal and historical trajectory; (2) an egocentric observation history $\mathcal{O}_t = \{x_0, \dots, x_t\}$, consisting of RGB frames $x_t \in \mathbb{R}^{H \times W \times 3}$, and (3) the robot state $s_t \in \mathbb{R}^3$, encoding ego-motion information such as linear velocity and angular velocity. Conditioned on these inputs, the agent must output an action $\mathbf{a}_t$ at each timestep to safely and efficiently progress toward the goal. The environment evolves in response to the agent’s actions, and the episode continues until the agent reaches the goal or terminates.

We consider settings where a large-scale VLM is employed to interpret the natural-language instructions and provide semantic guidance. While such models are powerful, their reasoning often introduces non-negligible inference latency Δt . As a result, semantic outputs may become temporally misaligned with the agent’s current observations and state, creating a key challenge for real-time navigation.

3.2. Think-in-Control VLA

An overview of the TIC-VLA framework is shown in Figure 2. TIC-VLA adopts a dual-system execution paradigm in which high-level semantic reasoning and low-level control operate asynchronously. Crucially, rather than treating this as an architectural contribution, we explicitly model the resulting inference delay as part of the control problem. The key design principle is to expose reasoning latency to the action policy and train the policy to act under delayed semantic observations. We employ InternVL3-1B (Zhu et al., 2025) as the vision-language backbone for semantic and instruction understanding. At a high level, a VLM performs semantic reasoning over delayed visual context and language instructions, while a reactive action policy executes at a high control frequency and never blocks on VLM inference. The action policy conditions on cached semantic representations together with explicit latency and ego-motion metadata, allowing it to interpret the delayed semantic information in the correct temporal context. This latency-aware semantic-control coupling enables robust navigation despite asynchronous and delayed reasoning updates.

VLM Semantic Reasoning. We formalize inference latency as a core variable in the system. We define the *effective reasoning latency* as $\Delta t = t_{\text{infer}} + t_{\text{elapse}} \geq 0$, which accounts for both the VLM inference time (t_{infer}) and the elapsed time since the last completed reasoning update (t_{elapse}). At the current timestep t , the VLM operates on visual observations anchored at time $t - \Delta t$, rather than the current frame. Given a set of historical frames $\mathcal{X}_{t-\Delta t}^{\text{vlm}} = \{x_{t-\Delta t-\delta} \mid \delta \in \{0, 3, 6, 9\}\}$, the VLM performs semantic reasoning conditioned on the instruction $\mathcal{I}$. The resulting output, denoted $\mathcal{R}_{t-\Delta t}$, encodes high-level scene understanding, critical object identification, intent prediction, and future target waypoints derived from delayed observations. The reasoning results, including predicted waypoints, are generated relative to the time of inference start $t - \Delta t$, rather than the current control timestep. This explicit temporal anchoring allows the downstream policy to know when the reasoning was produced, rather than treating the reasoning results as instantaneous or noisy signals.

Latency-Aware Action Policy. The action policy π_θ runs at a high frequency and is responsible for real-time planning. At each timestep t , it conditions on four categories of inputs: (1) the current visual observation x_t ; (2) the current robot state s_t ; (3) the most recent semantic hidden state $\mathcal{S}_{t-\Delta t}$ produced by the VLM (i.e., the last-layer key-value cache); (4) explicit latency metadata, including the effective latency Δt and the corresponding motion offsets $\Delta \mathbf{p}_t = (\Delta x, \Delta y, \Delta \theta)$ accumulated since reasoning generation. Providing both delayed semantic states and latency metadata establishes a *delayed semantic-control interface*: semantic features describe a past state of the world, while the control policy

is responsible for reinterpreting them in the current frame. This allows the policy to reason consistently about delayed semantics as the robot moves during inference. The policy outputs a short horizon of future actions:

$$\mathbf{a}_t = \{a_t^1, \dots, a_t^T\} = \pi_\theta(\mathcal{S}_{t-\Delta t}, x_t, s_t, \Delta t, \Delta \mathbf{p}_t), \quad (1)$$

where each $a_t^i \in \mathbb{R}^3$ represents a continuous action. The action chunks are integrated into a short-horizon trajectory, and a target point is chosen for execution.

As shown in Figure 3(a), the action policy utilizes a dedicated action query token that attends to the scene context through a stack of cross-attention Transformer layers. Visual tokens and VLM cache features are first projected into a shared latent space via MLP layers, while the robot state and latency metadata are encoded and added with positional embeddings. These inputs are concatenated as key-value tokens, and the updated action query representation is passed through an MLP to generate the action outputs.

Asynchronous Semantic Reasoning and Control. TIC-VLA operates in a closed-loop asynchronous manner. The VLM periodically updates semantic reasoning based on delayed visual inputs, while the action policy continuously executes without waiting for inference to complete. The cached VLM hidden state is updated as follows:

$$(\mathcal{S}_t^{\text{cache}}, \mathcal{R}_t^{\text{cache}}) = \begin{cases} (\mathcal{S}_t, \mathcal{R}_t), & \text{if inference finishes at } t, \\ (\mathcal{S}_{t^-}^{\text{cache}}, \mathcal{R}_{t^-}^{\text{cache}}), & \text{otherwise,} \end{cases} \quad (2)$$

where t^- denotes the most recent timestep prior to t at which inference was completed. At every control timestep, the action policy conditions on the most recent cached VLM hidden state $\mathcal{S}_{t-\Delta t} = \mathcal{S}_t^{\text{cache}}$. In addition, by conditioning action generation on latency metadata and ego-motion, TIC-VLA can reinterpret stale semantic information in the current state. An illustration of the asynchronous inference process is shown in Figure 3(d), and latency is measured as the sum of two parts: VLM reasoning inference time and the elapsed time since the last finished inference.

3.3. Latency-Consistent Training Pipeline

We adopt a three-stage training pipeline designed to enforce *latency consistency* between training and deployment. An overview of the pipeline is provided in Figure 3(c).

Supervised Fine-tuning of the VLM. We first fine-tune the VLM on structured semantic reasoning data collected from both simulation and real-world environments. Training samples are automatically annotated using GPT-5. Given past and future image sequences, the robot’s positional context, and the corresponding trajectory, GPT-5 produces: (1) a long-horizon instruction describing the navigation goal, and (2) a concise, structured reasoning trace capturing critical object identification, intention prediction, and resulting action. Details are provided in the supplementary material.

During this stage, the vision encoder is kept frozen. The language model is trained to produce reasoning tokens and waypoint predictions conditioned on visual tokens and instructions. We optimize the standard autoregressive cross-entropy loss over the target token sequence:

$$\mathcal{L}_l = -\frac{1}{N_l} \sum_{t=1}^{N_l} \log p_\phi(y_t | y_{<t}, \mathcal{I}, \mathbf{V}), \quad (3)$$

where y_t denotes the t -th token in the target sequence, N_l is the sequence length, and ϕ are the language model parameters. We mix waypoint-only and scene-reasoning-augmented targets during training for flexible prompting at inference time.

Imitation Learning under Reasoning Latency. To compensate for the uncertain delay in semantic reasoning, we perturb originally aligned and synchronous demonstrations to synthesize training data with delayed semantic reasoning. Specifically, we sample reasoning delays Δt uniformly from $[0, 10]$ seconds and condition the policy on: (1) the current image input and robot state, (2) KV cache features from the delayed VLM reasoning output, (3) explicit latency metadata. This exposes the policy to a range of temporally misaligned semantic representations during training. The action policy π_θ is trained via imitation learning using human demonstration trajectories. As low-level control actions are not available, we integrate the predicted actions forward to obtain positions (x, y, θ) over the prediction horizon and compare them against ground-truth trajectories using a smooth L_1 loss:

$$\mathcal{L}_a = \frac{1}{T} \sum_{i=1}^T \text{SmoothL1}(\hat{p}_t^{(i)} - p_t^{(i)}), \quad (4)$$

where $\hat{p}_t^{(i)}$ is the predicted pose at sub-timestep i , and $p_t^{(i)}$ is the ground-truth pose.

Reinforcement Learning with Asynchronous Guidance. While imitation learning with synthesized reasoning delays provides a strong initialization that accounts for inference latency, the resulting training data distribution remains mismatched to the closed-loop distribution induced by coupled, asynchronous reasoning-action interactions. Motivated by prior work (Lu et al., 2025; Li et al., 2025; Chen et al., 2025a), we fine-tune only the action policy using RL while keeping the vision encoder and language model frozen. This allows the policy to learn to interpret delayed VLM guidance, handle dynamic agents, and mitigate variability introduced by asynchronous inference.

We construct a simulation environment with dynamic human agents and train the policy using Proximal Policy Optimization (PPO) (Schulman et al., 2017). The value network, shown in Figure 3(b), takes as input the current image tokens, the goal position, and the robot state, and outputs the

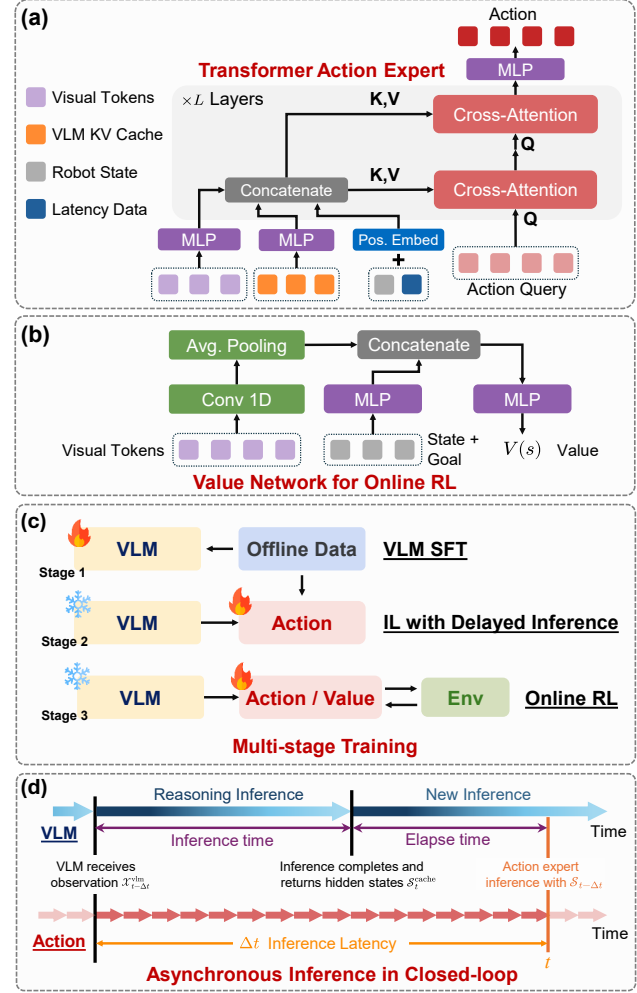


Figure 3. Details of TIC-VLA action policy structure, training, and asynchronous execution. (a) Latency-aware action policy that predicts action chunks from multimodal inputs. (b) Value network used during online reinforcement learning. (c) Three-stage latency-consistent training pipeline combining VLM supervision, imitation learning, and reinforcement learning. (d) Asynchronous inference and control with explicit latency modeling.

estimated state value. The policy outputs a Gaussian action distribution, where the action derived from the predicted trajectory is the mean, and the standard deviation is learned during training. The reward function is defined as:

$$r_t = w_g r_t^{\text{goal}} + w_p r_t^{\text{progress}} + w_c r_t^{\text{collision}} + w_s r_t^{\text{speed}}, \quad (5)$$

where r_t^{goal} rewards reaching the target, r_t^{progress} encourages progress toward the goal, $r_t^{\text{collision}}$ penalizes collisions with humans or static obstacles, and r_t^{speed} penalizes both excessively slow motion and excessive speed. w_g , w_p , w_c , and w_s are weights for these terms.

To further improve robustness under asynchronous deployment, we inject stochastic inference delays following each VLM update to mimic the latency characteristics observed on edge hardware. This enforces consistency between train-

ing and execution conditions. Additional implementation details are provided in the supplementary material.

3.4. DynaNav

Evaluating our proposed latency-aware VLA framework requires more realistic navigation benchmarks than classic VLN benchmarks such as R2R (Li et al., 2019), VLN-CE (Krantz et al., 2020), and RxR (Ku et al., 2020), which operate in small indoor environments and abstract navigation as viewpoint transitions without physical interaction. VLN-PE (Wang et al., 2025a) and GRUtopia (Wang et al., 2024) support embodied evaluation but do not model navigation among dynamic human participants, while SocialHM3D (Gong et al., 2025) and HA-VLN (Dong et al., 2025) incorporate human agents with limited visual realism. To fill this gap, DynaNav provides a realistic benchmark integrating language-guided navigation, large-scale scenes, diverse human agents, and physics-based robot control.

Simulation Environments. We construct simulation environments in Isaac Sim (Isaac-sim development team, 2025), with realistic, controllable dynamic interactions. Four representative scenes (i.e., warehouse, hospital, office, and outdoor sidewalk) are designed to capture a broad range of navigation contexts. Human behaviors are modeled using Isaac Sim’s built-in human simulation tools, supporting behaviorally plausible pedestrian movement. Our simulation setup supports both wheeled (Nova Carter) and quadruped (Boston Dynamics Spot) robots. We develop custom robot behavior scripts that allow two modes of operation: (1) Human teleoperation mode, which enables manual control for collecting expert demonstrations, and (2) End-to-end model control mode, which allows direct control of the robot with end-to-end planning models.

Designed for Scalable RL Training. We employ Isaac Lab (Mittal et al., 2025) and leverage its GPU-accelerated simulation to build environments for scalable reinforcement learning training. We implement a custom human behavior control script to generate human movements within the environment. Human-robot and robot-scene interactions are fully physics-based with realistic contact dynamics. This setup allows us to run multiple parallel environments, enabling scalable and efficient RL training.

Diverse Benchmark Tasks. We develop a comprehensive benchmark of 85 test cases to evaluate navigation performance across diverse conditions. Tasks vary along three dimensions: (1) *Crowd Density*, ranging from empty spaces to densely populated settings, capturing different levels of dynamic complexity. (2) *Navigation Distance*, adjusted from short-term navigation to long-horizon planning, reflecting increasing navigation difficulty. (3) *Scene Type*, with evaluation conducted across four distinct environments to assess robustness to varying spatial layouts and human

behaviors. For each task, standardized initial and goal positions are specified with a language instruction. Additional details are provided in the supplementary material.

4. Experiments

4.1. Experimental Setup

Datasets. We train the model using three datasets featuring dynamic human-robot interactions: (1) SCAND (Karnan et al., 2022), which contains 8.7 hours of robot-driven trajectories across diverse social environments; (2) GND (Liang et al., 2025), which comprises over 11 hours of recorded data collected in various campus environments; (3) DynaNav simulation dataset, collected using our designed dynamic simulation environments, containing 5.1 hours of robot navigation data across multiple scene types.

Implementation Details. For VLM SFT, we employ full-parameter training due to the compact size of TIC-VLA. Training is performed using Distributed Data Parallel on eight NVIDIA L40S GPUs, with a batch size of 2 per GPU. AdamW is used as the optimizer with a cosine learning rate schedule, initialized at 2×10^{-5} . For training the action expert, we increase the batch size to 16 per GPU and set the initial learning rate to 2×10^{-4} . RL fine-tuning of the action policy is conducted on a single NVIDIA L40S GPU for 400 iterations across three tasks in three environments. Additional details, including hyperparameters and data pre-processing, are provided in the supplementary material.

Baselines. We evaluate TIC-VLA against two categories of methods. Point-goal navigation policies are included as reference baselines to contextualize task difficulty: (1) a vanilla Behavior Cloning (BC) policy that maps RGB observations and point-goal commands directly to actions; (2) a vanilla RL policy trained on RGB observations and point-goal commands; (3) NavDP (Cai et al., 2025), a point-goal-conditioned diffusion-based navigation policy. The primary language-guided navigation baselines are listed below: (1) NaVILA (Cheng et al., 2024), a hierarchical VLA model that translates language instructions into mid-level commands; (2) Uni-NaVid (Zhang et al., 2024a), a unified video-based VLA model trained across multiple navigation tasks; (3) DualVLN (Wei et al., 2025a), a dual-system VLA model for language-guided navigation; (4) MobileVLA-R1 (Huang et al., 2025), a reasoning-enhanced VLA framework for mobile robots with chain-of-thought supervision; (5) OmniVLA (Hirose et al., 2025), an omni-modal VLA navigation model that supports language, visual-goal, and pose-goal conditioning. All language-guided baselines are fine-tuned on the same training datasets and evaluated under the same settings for a fair comparison.

Evaluation metrics. We adopt the following evaluation metrics: (1) Navigation Error (NE): the final distance between

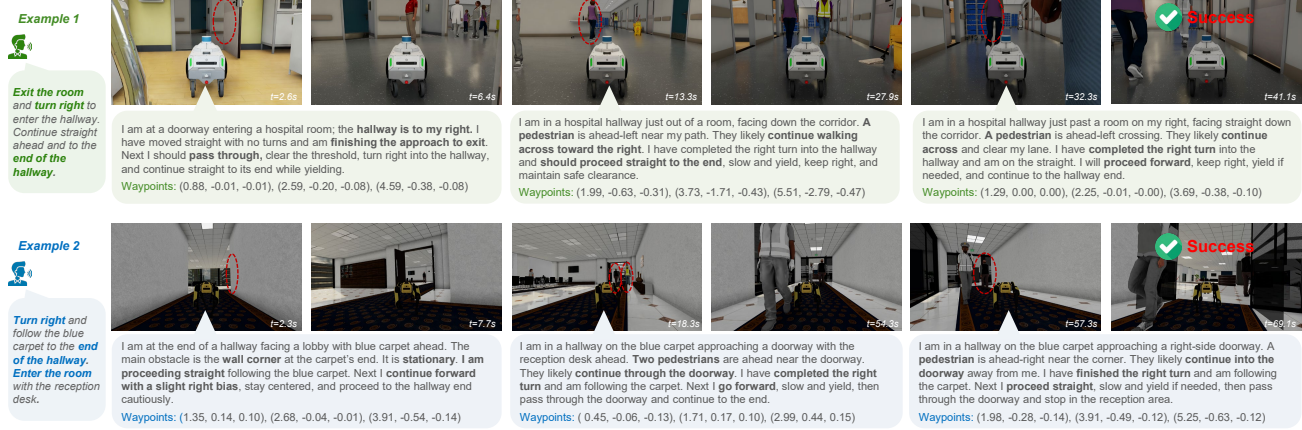


Figure 4. Qualitative closed-loop results of TIC-VLA in DynaNav hospital (top) and office (bottom) environments. TIC-VLA demonstrates effective semantic reasoning while producing reactive navigation actions in dynamic scenarios.

the agent and the goal; (2) Success Rate (SR): the percentage of episodes in which the agent stops within 1.5 meters of the goal; (3) Success weighted by Path Length (SPL): SR weighted by the ratio between the shortest path length and the actual path length, penalizing inefficient trajectories; (4) Collision Rate (CR): the percentage of episodes in which the agent collides with static obstacles or humans, quantifying the safety of navigation behavior.

Real-world experimental setup. To evaluate real-time performance on edge devices, we deploy the model on two platforms with different power budgets: an NVIDIA Jetson Orin NX (25W) and an RTX 4060 Laptop GPU (50W), representing typical edge computing capabilities. An RTX A6000 GPU is used only when the baselines cannot run on these devices. The navigation policy is executed on a Unitree Go2 quadruped robot in real-world navigation tasks. We employ FlashAttention to improve inference efficiency. Performance is measured by the average success rate. An episode is considered a failure if manual intervention is required to prevent collisions.

4.2. Simulation Testing

Performance on the DynaNav benchmark. All experiments are conducted on an NVIDIA L40S GPU, with the action policy running at 10 Hz and asynchronous VLM reasoning running at 0.5 Hz. Results are summarized in Table 1. Point-goal methods bypass vision-language inference and therefore incur no reasoning latency, but they also rely on privileged goal information. In contrast, TIC-VLA uses only egocentric observations and language instructions, without access to privileged goals or maps. TIC-VLA achieves strong performance under this setting. Without RL fine-tuning, TIC-VLA is competitive with NavDP, a point-goal method with privileged state access, and outperforms the vanilla BC and RL baselines. After RL fine-tuning, TIC-VLA achieves the highest success rate and the lowest col-

lision rate, indicating improved closed-loop robustness in dynamic scenes. Although NavDP obtains the lowest navigation error and the highest SPL, it benefits from direct point-goal supervision. TIC-VLA also clearly outperforms prior VLN baselines, including Uni-NaVid, NaVILA, and DualVLN. Their lower success rates and higher collision rates suggest that methods designed for more abstract navigation settings struggle with continuous control and dynamic obstacles. Moreover, TIC-VLA outperforms stronger VLA baselines, including OmniVLA and MobileVLA. The synchronous TIC-VLA variant also degrades substantially, confirming that blocking control on slow VLM inference harms real-time navigation. Figure 4 shows qualitative examples of TIC-VLA’s semantic reasoning and reactive navigation in dynamic environments. We provide additional dynamic video results on the [project website](#).

Table 1. Performance of TIC-VLA and baseline methods on the DynaNav benchmark. BC, RL, and NavDP are point-goal-based.

Method	NE (↓)	SR (↑)	SPL (↑)	CR (↓)
BC Policy	9.96	45.88	41.52	35.29
RL Policy	12.20	30.59	28.45	36.47
NavDP	8.61	54.12	52.62	30.59
Uni-NaVid	15.90	22.35	19.61	49.41
NaVILA	17.20	28.24	25.51	48.24
DualVLN	16.45	30.59	27.82	47.06
OmniVLA	16.53	31.76	28.33	49.41
MobileVLA	16.09	32.94	30.31	45.88
TIC-VLA (Sync.)	16.31	32.94	29.64	41.18
TIC-VLA (no RL)	10.85	47.06	42.41	34.12
TIC-VLA	10.55	55.29	50.29	28.24

Latency Robustness Analysis. Figure 5 reports performance under increasing VLM reasoning inference latency for TIC-VLA in simulation, before and after RL fine-tuning. As latency increases, the IL-based action expert exhibits a noticeable decline in success rate, indicating higher sensitivity to delayed reasoning updates. In contrast, the RL-

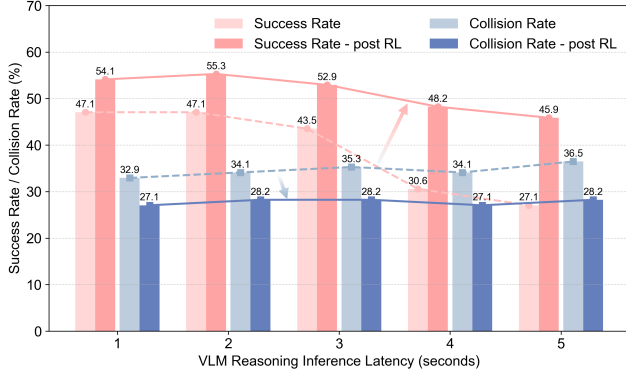


Figure 5. The effect of VLM asynchronous reasoning inference latency in TIC-VLA on task performance.

fine-tuned policy maintains consistently higher success rates across all latency settings, demonstrating improved robustness to inference latency. This result highlights the effectiveness of RL fine-tuning in mitigating latency-induced performance degradation. Collision rates are relatively insensitive to inference latency, suggesting that the asynchronous policy preserves reactivity independent of reasoning speed.

Influence of Semantic-Control Interface and Latency Awareness. We evaluate the impact of different delayed semantic-control interfaces and latency-awareness on TIC-VLA. Specifically, we compare interface variants that use waypoint-based guidance and KV-cache-based features, each trained with and without explicit latency-aware modeling and training. As shown in Table 2, using KV-cache features significantly improves navigation success, and latency-awareness enhances performance under asynchronous inference. The waypoint-based interface leads to inferior performance due to its sparsity and potential inconsistency with the agent’s local observations. In contrast, KV-cache features preserve richer semantic and contextual information from the VLM, allowing the control policy to better adapt to delayed reasoning outputs. Combining both the KV cache feature interface and latency-aware modeling and training achieves the best overall performance.

Table 2. Influence of semantic interface and latency training.

Interface	Latency	NE ($\downarrow$)	SR ($\uparrow$)	SPL ($\uparrow$)	CR ($\downarrow$)
Waypoint	×	21.17	16.47	15.89	47.06
Waypoint	✓	20.32	22.35	18.34	42.35
KV Cache	×	16.74	30.59	28.31	40.00
KV Cache	✓	10.85	47.06	42.41	34.12

4.3. Real-world Testing

We evaluate TIC-VLA on a Unitree Go2 robot across four real-world navigation tasks: (1) an indoor hallway with dynamic human and static obstacles, (2) a cluttered office, (3) an outdoor plaza, and (4) an outdoor walkway with uneven terrain. These scenarios cover indoor and outdoor deploy-

ment conditions with challenges such as moving pedestrians, narrow passages, clutter, and terrain variation. Task descriptions and hardware configurations are shown in Figure 6. All evaluations are conducted zero-shot, without task-specific training data. For each task, we conduct five trials and report the average success rate. During deployment, RGB images from the front-facing camera are streamed to TIC-VLA, where VLM reasoning runs asynchronously from the high-frequency action policy.

Table 3. Real-world testing results. Runtimes for dual-system methods are reported as action policy / VLM reasoning latency.

Method	Platform	Success Rate ($\uparrow$)	Runtime (ms)
TIC-VLA (no RL)	4060	0.70	–
TIC-VLA	4060	0.85	85.73 / 3430.73
TIC-VLA	Orin NX	0.75	120.27 / 4831.73
TIC-VLA	A6000	0.80	32.70 / 1681.66
Dual-VLN (7B)	A6000	0.50	299.92 / 1534.67
NaVILA (7B)	A6000	0.35	4106.62

We compare TIC-VLA with DualVLN (Wei et al., 2025a) and NaVILA (Cheng et al., 2024) under the same real-world settings. As shown in Table 3, TIC-VLA achieves the highest success rate with a smaller model and lower control-side latency. RL fine-tuning further improves success from 0.70 to 0.85, indicating stronger robustness in physical deployment, especially around pedestrians and obstacles. Across compute platforms, TIC-VLA maintains a 0.75 success rate on the Jetson Orin NX despite multi-second VLM reasoning latency, demonstrating its suitability for edge deployment. Although the A6000 reduces VLM latency, its success rate does not exceed local RTX 4060 deployment, likely due to remote communication overhead. These results show that real-world performance depends not only on inference speed, but also on communication latency and closed-loop control frequency. Overall, TIC-VLA transfers zero-shot to real-world robot navigation and remains robust under asynchronous high-latency VLM inference, demonstrating the practical value of explicit latency modeling.

4.4. Ablation Study

Influence of Reasoning at Test Time. We evaluate explicit VLM reasoning during inference by comparing the RL fine-tuned model with and without reasoning-token outputs. As shown in Table 4, enabling reasoning clearly improves navigation performance, reducing NE from 14.23 to 10.55 while increasing SR from 40.00 to 55.29 and SPL from 34.22 to 50.29. Disabling reasoning reduces VLM overhead and increases the forward rate from 0.5 Hz to 4 Hz, but leads to weaker goal completion and navigation progress. Although the non-reasoning variant has a lower collision rate, this mainly reflects reduced activity and more frequent failure rather than safer navigation. In contrast, the reasoning-enabled model interacts more actively with the environment while maintaining substantially higher task completion and

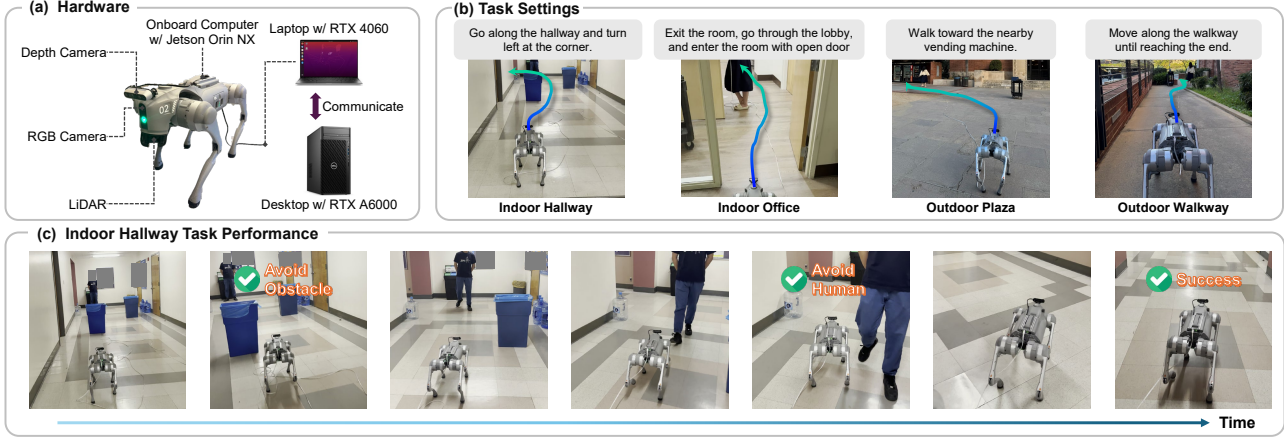


Figure 6. Real-world evaluation of TIC-VLA. (a) Hardware configuration, including the robot platform and computation setup. (b) Designed indoor and outdoor vision-language navigation tasks. (c) Qualitative results from an indoor hallway navigation task, showing the robot following natural language instructions while avoiding obstacles and humans and reaching the goal.

navigation efficiency. These results indicate that test-time reasoning improves task success, while TIC-VLA mitigates its latency through explicit latency-aware control.

Table 4. Effect of VLM reasoning at test time.

Inference	NE ($\downarrow$)	SR ($\uparrow$)	SPL ($\uparrow$)	CR ($\downarrow$)
W/o Reasoning	14.23	40.00	34.22	25.88
W/ Reasoning	10.55	55.29	50.29	28.24

Influence of Action Prediction Horizon. We ablate the action prediction horizon to study the trade-off between short-term responsiveness and longer-horizon planning. A shorter horizon makes the policy more reactive but provides less planning context, whereas a longer horizon provides more temporal context but can make near-term action prediction less accurate. As shown in Table 5, the 3-second horizon achieves the best overall performance among TIC-VLA variants without RL fine-tuning. The 1-second horizon obtains the lowest collision rate, suggesting stronger short-term reactivity, but yields a lower success rate and SPL. The 5-second horizon also underperforms the 3-second setting, indicating that overly long prediction horizons may reduce control accuracy. We therefore use a 3-second prediction horizon as the default setting.

Table 5. Effect of action prediction horizon. Results are reported without RL fine-tuning.

Horizon	NE ($\downarrow$)	SR ($\uparrow$)	SPL ($\uparrow$)	CR ($\downarrow$)
1s	12.23	42.35	38.72	32.94
3s (ours)	10.85	47.06	42.41	34.12
5s	11.77	40.00	36.46	35.29

Influence of Ego-motion Offset in Action Policy. Table 6 shows the effect of incorporating ego-motion offset into the latency-aware action policy. Without ego-motion offset, the policy receives delayed semantic guidance but lacks explicit information about how far the robot has moved since the

guidance was generated. As a result, stale VLM features may be treated as temporally aligned with the current observation, leading to degraded navigation accuracy, lower success rates, and less efficient trajectories. By explicitly modeling the ego-motion accumulated during the inference delay, the policy can reinterpret delayed semantic features in the current robot frame. This improves the alignment between semantic reasoning and real-time control, allowing the action expert to compensate for robot displacement and heading changes during asynchronous VLM inference, resulting in substantially improved performance.

Table 6. Effect of incorporating ego-motion offset into the latency-aware action policy. Results are reported without RL fine-tuning.

Method	NE ($\downarrow$)	SR ($\uparrow$)	SPL ($\uparrow$)	CR ($\downarrow$)
W/o Motion Offset	12.97	41.18	36.36	36.47
W/ Motion Offset	10.85	47.06	42.41	34.12

5. Conclusions

We introduce TIC-VLA, a latency-aware VLA framework that addresses the temporal mismatch between slow semantic reasoning and real-time control. By using a delayed semantic-control interface and training policies under realistic inference delays, TIC-VLA enables robust language-guided navigation under substantial latency. Simulation and real-world results show consistent improvements over prior navigation and language-guided baselines.

Limitations. TIC-VLA has three main limitations. First, the current system is not yet fully optimized for runtime efficiency, leaving room for faster inference and deployment. Second, the real-world evaluation remains limited in scale, requiring larger studies to further validate robustness. Third, extending beyond navigation to domains such as robotic manipulation remains future work.

Acknowledgements

This work was supported by the Center of Excellence on New Mobility and Automated Vehicles (Mobility COE), the National Science Foundation (NSF) under Award No. 2346267, POSE: Phase II - DriveX: An Open Source Ecosystem for Automated Driving and Intelligent Transportation Research, and the NVIDIA Academic Grant Program. Yun Zhang was supported by the Amazon Trainium Fellowship.

Impact Statement

This work advances VLA models for real-time robot navigation by addressing inference latency and limited onboard computation. TIC-VLA may support deployment in service robotics, logistics, and assisted mobility, especially on edge platforms. At the same time, robots operating in shared human spaces can pose safety and ethical risks, including misinterpretation of instructions, collisions, and socially inappropriate behavior. Although latency-aware reactive control helps reduce these risks, the system remains subject to perception, reasoning, and control failures. Human oversight is therefore necessary in safety-critical settings.

References

- Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., et al. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- Bar, A., Zhou, G., Tran, D., Darrell, T., and LeCun, Y. Navigation world models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 15791–15801, 2025.
- Cai, W., Peng, J., Yang, Y., Zhang, Y., Wei, M., Wang, H., Chen, Y., Wang, T., and Pang, J. Navdp: Learning sim-to-real navigation diffusion policy with privileged information guidance. *arXiv preprint arXiv:2505.08712*, 2025.
- Castro, M. G., Rajagopal, S., Gorbato, D., Schmitt, M., Baijal, R., Zhang, O., Scalise, R., Talia, S., Romig, E., de Melo, C., et al. Vamos: A hierarchical vision-language-action model for capability-modulated and steerable navigation. *arXiv preprint arXiv:2510.20818*, 2025.
- Chandaka, B., Wang, G. X., Chen, H., Che, H., Zhai, A. J., and Wang, S. Human-like navigation in a world built for humans. In *Conference on Robot Learning*, pp. 1790–1808. PMLR, 2025.
- Chen, H., Zhao, M., Yang, R., Ma, Q., Yang, K., Yao, J., Wang, K., Bai, H., Wang, Z., Pan, R., et al. Era: Transforming vlms into embodied agents via embodied prior learning and online reinforcement learning. *arXiv preprint arXiv:2510.12693*, 2025a.
- Chen, Z., Guo, Y., Chu, Z., Luo, M., Shen, Y., Sun, M., Hu, J., Xie, S., Yang, K., Shi, P., et al. Socialnav: Training human-inspired foundation model for socially-aware embodied navigation. *arXiv preprint arXiv:2511.21135*, 2025b.
- Cheng, A.-C., Ji, Y., Yang, Z., Gongye, Z., Zou, X., Kautz, J., Büyük, E., Yin, H., Liu, S., and Wang, X. Navila: Legged robot vision-language-action model for navigation. *arXiv preprint arXiv:2412.04453*, 2024.
- Choi, W., Kim, W. K., Yoo, M., and Woo, H. Embodied cot distillation from llm to off-the-shelf agents. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 8702–8721, 2024.
- Dong, Y., Wu, F., He, Q., Li, H., Li, M., Cheng, Z., Zhou, Y., Sun, J., Dai, Q., Cheng, Z.-Q., et al. Ha-vln: A benchmark for human-aware navigation in discrete-continuous environments with dynamic multi-human interactions, real-world validation, and an open leaderboard. *arXiv preprint arXiv:2503.14229*, 2025.
- Driess, D., Springenberg, J., Ichter, B., Yu, L., Li-Bell, A., Pertsch, K., Ren, A., Walke, H., Vuong, Q., Shi, L. X., et al. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. *Advances in Neural Information Processing Systems*, 38:102867–102888, 2026.
- Du, Y., Fu, T., Chen, Z., Li, B., Su, S., Zhao, Z., and Wang, C. VI-nav: real-time vision-language navigation with spatial reasoning. *arXiv preprint arXiv:2502.00931*, 2025.
- Eftekhari, A., Hendrix, R., Weihs, L., Duan, J., Caglar, E., Salvador, J., Herrasti, A., Han, W., VanderBil, E., Kembhavi, A., et al. The one ring: a robotic indoor navigation generalist. *arXiv preprint arXiv:2412.14401*, 2024.
- Elnoor, M., Weerakoon, K., Seneviratne, G., Liang, J., Rajagopal, V., and Manocha, D. Vilam: Distilling vision-language reasoning into attention maps for social robot navigation. *arXiv preprint arXiv:2503.09820*, 2026.
- Gao, C., Jin, L., Peng, X., Zhang, J., Deng, Y., Li, A., Wang, H., and Liu, S. Octonav: Towards generalist embodied navigation. *arXiv preprint arXiv:2506.09839*, 2025.
- Gong, Z., Hu, T., Qiu, R., and Liang, J. From cognition to precognition: A future-aware framework for social navigation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9122–9129. IEEE, 2025.
- He, H., Ma, Y., Wu, W., and Zhou, B. From seeing to experiencing: Scaling navigation foundation models with reinforcement learning. *arXiv preprint arXiv:2507.22028*, 2025.

- Hirose, N., Glossop, C., Shah, D., and Levine, S. Omnivla: An omni-modal vision-language-action model for robot navigation. *arXiv preprint arXiv:2509.19480*, 2025.
- Hu, J., Chen, J., Bai, H., Luo, M., Xie, S., Chen, Z., Liu, F., Chu, Z., Xue, X., Ren, B., et al. Astranav-world: World model for foresight control and consistency. *arXiv preprint arXiv:2512.21714*, 2025a.
- Hu, Z., Tang, C., Munje, M. J., Zhu, Y., Liu, A., Liu, S., Warnell, G., Stone, P., and Biswas, J. Composable-nav: Instruction-following navigation in dynamic environments via composable diffusion. In *Conference on Robot Learning*, pp. 4246–4268. PMLR, 2025b.
- Huang, T., Li, D., Yang, R., Zhang, Z., Yang, Z., and Tang, H. Mobilevla-rl: Reinforcing vision-language-action for mobile robots. *arXiv preprint arXiv:2511.17889*, 2025.
- InternRobotics. Internvla-n1: An open dual-system navigation foundation model with learned latent plans. <https://huggingface.co/InternRobotics/InternVLA-N1>, 2025. Accessed: 2025-10-10.
- Isaac-sim development team. IsaacSim: An open-source robotics simulation platform on NVIDIA Omniverse. <https://github.com/isaac-sim/IsaacSim>, 2025. Accessed: 2025-10-09; version v5.0.0.
- Karnan, H., Nair, A., Xiao, X., Warnell, G., Pirk, S., Toshev, A., Hart, J., Biswas, J., and Stone, P. Socially compliant navigation dataset (scand): A large-scale dataset of demonstrations for social navigation. *IEEE Robotics and Automation Letters*, 7(4):11807–11814, 2022.
- Krantz, J., Wijmans, E., Majundar, A., Batra, D., and Lee, S. Beyond the nav-graph: Vision and language navigation in continuous environments. In *European Conference on Computer Vision (ECCV)*, 2020.
- Ku, A., Anderson, P., Patel, R., Ie, E., and Baldrige, J. Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4392–4412, 2020.
- Li, H., Zuo, Y., Yu, J., Zhang, Y., Yang, Z., Zhang, K., Zhu, X., Zhang, Y., Chen, T., Cui, G., et al. Simplevla-rl: Scaling vla training via reinforcement learning. *arXiv preprint arXiv:2509.09674*, 2025.
- Li, X., Li, C., Xia, Q., Bisk, Y., Celikyilmaz, A., Gao, J., Smith, N. A., and Choi, Y. Robust navigation with language pretraining and stochastic sampling. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 1494–1499, 2019.
- Liang, J., Das, D., Song, D., Shuvo, M. N. H., Durrani, M., Taranath, K., Penskiy, I., Manocha, D., and Xiao, X. Gnd: Global navigation dataset with multi-modal perception and multi-category traversability in outdoor campus environments. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2383–2390. IEEE, 2025.
- Lin, F., Nai, R., Hu, Y., You, J., Zhao, J., and Gao, Y. Onetwovla: A unified vision-language-action model with adaptive reasoning. *arXiv preprint arXiv:2505.11917*, 2025.
- Liu, W., Zhao, H., Li, C., Biswas, J., Okal, B., Goyal, P., Chang, Y., and Pouya, S. X-mobility: End-to-end generalizable navigation via world modeling. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 7569–7576. IEEE, 2025a.
- Liu, W., Zhao, H., Li, C., Biswas, J., Pouya, S., and Chang, Y. Compass: Cross-embodiment mobility policy via residual rl and skill synthesis. *arXiv preprint arXiv:2502.16372*, 2025b.
- Lu, G., Guo, W., Zhang, C., Zhou, Y., Jiang, H., Gao, Z., Tang, Y., and Wang, Z. Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint arXiv:2505.18719*, 2025.
- Mittal, M., Roth, P., Tigue, J., Richard, A., Zhang, O., Du, P., Serrano-Muñoz, A., Yao, X., Zurbrugg, R., Rudin, N., et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.
- Munje, M. J., Tang, C., Liu, S., Hu, Z., Zhu, Y., Cui, J., Warnell, G., Biswas, J., and Stone, P. Socialnav-sub: Benchmarking vlms for scene understanding in social robot navigation. In *Conference on Robot Learning*, pp. 1120–1143. PMLR, 2025.
- Payandeh, A., Pokhrel, A., Song, D., Zampieri, M., and Xiao, X. Narrate2nav: Real-time visual navigation with implicit language reasoning in human-centric environments. *arXiv preprint arXiv:2506.14233*, 2025a.
- Payandeh, A., Song, D., Nazeri, M., Liang, J., Mukherjee, P., Raj, A. H., Kong, Y., Manocha, D., and Xiao, X. Socialllava: Enhancing social robot navigation through human-language reasoning. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 17192–17198. IEEE, 2025b.

- Raychaudhuri, S. and Chang, A. X. Semantic mapping in indoor embodied ai-a survey on advances, challenges, and future directions. *Transactions on Machine Learning Research*, 2025.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shcherbyna, V., Kastner, L., Diaz, D., Nguyen, H. G., Schreff, M. H.-K., Seeger, T., Kreutz, J., Martban, A., Shen, Z., Zeng, H., et al. Arena 4.0: A comprehensive ros2 development and benchmarking platform for human-centric navigation using generative-model-based environment generation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9138–9144. IEEE, 2025.
- Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zouitine, A., Aractingi, M., Pascal, C., Russi, M., Marafioti, A., et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- Tang, C., Abbatematteo, B., Hu, J., Chandra, R., Martín-Martín, R., and Stone, P. Deep reinforcement learning for robotics: A survey of real-world successes. *Annual Review of Control, Robotics, and Autonomous Systems*, 8 (1):153–188, 2025.
- Wang, H., Chen, J., Huang, W., Ben, Q., Wang, T., Mi, B., Huang, T., Zhao, S., Chen, Y., Yang, S., et al. Grutopia: Dream general robots in a city at scale. *arXiv preprint arXiv:2407.10943*, 2024.
- Wang, L., Xia, X., Zhao, H., Wang, H., Wang, T., Chen, Y., Liu, C., Chen, Q., and Pang, J. Rethinking the embodied gap in vision-and-language navigation: A holistic study of physical and visual disparities. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9455–9465, 2025a.
- Wang, S., Zhang, J., Li, M., Liu, J., Li, A., Wu, K., Zhong, F., Yu, J., Zhang, Z., and Wang, H. Trackvla: Embodied visual tracking in the wild. In *Conference on Robot Learning*, pp. 4139–4164. PMLR, 2025b.
- Wei, M., Wan, C., Peng, J., Yu, X., Yang, Y., Feng, D., Cai, W., Zhu, C., Wang, T., Pang, J., et al. Ground slow, move fast: A dual-system foundation model for generalizable vision-and-language navigation. *arXiv preprint arXiv:2512.08186*, 2025a.
- Wei, M., Wan, C., Yu, X., Wang, T., Yang, Y., Mao, X., Zhu, C., Cai, W., Wang, H., Chen, Y., et al. Streamvln: Streaming vision-and-language navigation via slowfast context modeling. *arXiv preprint arXiv:2507.05240*, 2025b.
- Wu, J., Zhou, Y., Yang, H., Huang, Z., and Lv, C. Human-guided reinforcement learning with sim-to-real transfer for autonomous navigation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(12):14745–14759, 2023.
- Wu, W., He, H., Zhang, C., He, J., Zhao, S. Z., Gong, R., Li, Q., and Zhou, B. Towards autonomous micromobility through scalable urban simulation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 27553–27563, 2025.
- Xu, Z., Chiang, H.-T. L., Fu, Z., Jacob, M. G., Zhang, T., Lee, T.-W. E., Yu, W., Schenck, C., Rendleman, D., Shah, D., et al. Mobility vla: Multimodal instruction navigation with long-context vlms and topological graphs. In *8th Annual Conference on Robot Learning*, 2024.
- Yu, Z., Long, Y., Yang, Z., Zeng, C., Fan, H., Zhang, J., and Dong, H. Correctnav: Self-correction flywheel empowers vision-language-action navigation model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 18737–18745, 2026.
- Yuan, M., Wang, L., and Waslander, S. L. Opennav: Open-world navigation with multimodal large language models. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 18948–18955. IEEE, 2025.
- Zeng, S., Qi, D., Chang, X., Xiong, F., Xie, S., Wu, X., Liang, S., Xu, M., and Wei, X. Janusvln: Decoupling semantics and spatiality with dual implicit memory for vision-language navigation. *arXiv preprint arXiv:2509.22548*, 2025.
- Zhang, J., Wang, K., Wang, S., Li, M., Liu, H., Wei, S., Wang, Z., Zhang, Z., and Wang, H. Uni-navid: A video-based vision-language-action model for unifying embodied navigation tasks. *arXiv preprint arXiv:2412.06224*, 2024a.
- Zhang, J., Wang, K., Xu, R., Zhou, G., Hong, Y., Fang, X., Wu, Q., Zhang, Z., and Wang, H. Navid: Video-based vlm plans the next step for vision-and-language navigation. *arXiv preprint arXiv:2402.15852*, 2024b.
- Zhang, J., Li, A., Qi, Y., Li, M., Liu, J., Wang, S., Liu, H., Zhou, G., Wu, Y., Li, X., et al. Embodied navigation foundation model. *arXiv preprint arXiv:2509.12129*, 2025a.
- Zhang, S., Qiao, Y., Wang, Q., Guo, L., Wei, Z., and Liu, J. Flexvln: Flexible adaptation for diverse vision-and-language navigation tasks. *IEEE Transactions on Multimedia*, 27:6307–6318, 2025b.

Zhao, Q., Lu, Y., Kim, M. J., Fu, Z., Zhang, Z., Wu, Y., Li, Z., Ma, Q., Han, S., Finn, C., et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 1702–1713, 2025.

Zhou, G., Hong, Y., and Wu, Q. Navgpt: Explicit reasoning in vision-and-language navigation with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 7641–7649, 2024.

Zhou, Z., Cai, T., Zhao, S., Zhang, Y., Huang, Z., Zhou, B., and Ma, J. Autovla: A vision-language-action model for end-to-end autonomous driving with adaptive reasoning and reinforcement fine-tuning. *Advances in Neural Information Processing Systems*, 38:27920–27956, 2026.

Zhu, J., Wang, W., Chen, Z., Liu, Z., Ye, S., Gu, L., Tian, H., Duan, Y., Su, W., Shao, J., et al. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models. *arXiv preprint arXiv:2504.10479*, 2025.

TIC-VLA Supplementary Materials

A. Model Details

VLM Backbone. Our model is built upon an *InternVL3-1B* vision-language model, which consists of an *InternViT-300M* vision encoder and a *Qwen2.5-0.5B* language model. The VLM processes a sequence of historical images spanning up to a nine-second temporal window to perform reasoning and generate high-level planning waypoints. Images are sampled at three-second intervals, resulting in three historical frames and one current frame. Each image is captured at a resolution of 1920×1080 , and these images are encoded into visual tokens of size 4×256 , which together provide both temporal and historical context for reasoning.

Due to reasoning inference latency, the visual observations fed into the VLM are temporally lagged relative to the real-time sensory inputs received by the action expert. To ensure consistent visual representations across modules, the vision encoder is shared between the VLM and the action expert. Specifically, the shared encoder processes the current image observation and outputs visual tokens that are directly consumed by the action expert for real-time control.

Action Expert. The action expert is implemented as a Transformer-based architecture composed of six cross-attention layers. Its inputs include: (1) current visual tokens (256 tokens) extracted from the shared vision encoder, (2) the robot’s proprioceptive state, (3) delayed semantic guidance from the VLM, and (4) latency-related metadata. The VLM guidance is provided either in the form of abstract waypoint predictions or as key-value (KV) cache features from the final Transformer layer of the VLM. The robot state is represented by a vector consisting of the linear velocities (v_x, v_y) and the yaw rate (ω_z).

To explicitly account for inference latency, we incorporate latency-related metadata, including the estimated robot displacement during the inference delay ($\Delta x, \Delta y, \Delta \theta$), and the corresponding time delay Δt . Visual tokens are projected from 896 dimensions to 512 dimensions, VLM KV cache features are projected from 256 to 512 dimensions, and low-dimensional inputs (waypoint guidance and latency metadata) are projected to 512 dimensions, followed by positional embeddings. We apply token dropout to the KV-cache tokens (drop rate 0.1) to encourage robustness to missing semantic context, and apply dropout to the encoded robot-state tokens/positional embeddings (drop rate 0.5) to reduce overfitting to state inputs.

All processed features are treated as scene context and serve as keys and values in the cross-attention Transformer. The queries are learnable embeddings indexed by time steps, enabling the model to produce structured predictions. Transformer outputs are passed through an MLP to produce action chunks at each predicted step.

We predict actions over a future horizon of three seconds, discretized into $T = 30$ action chunks. Each action is parameterized as (d_x, d_y, d_θ) , representing relative motion increments. A simple integrator (cumulative sum) is applied to accumulate these actions into a continuous trajectory, yielding waypoints of the form (x, y, θ) .

Value Network. The value network shares the same vision encoder as the action expert and operates on the current image observation. The resulting 256 visual tokens are first processed by three 1D convolutional layers followed by an average pooling layer, reducing them to a compact feature representation. In parallel, the robot state (v_x, v_y) and yaw rate (ω_z), as well as the relative goal position, are concatenated and projected into the same feature dimension using MLPs. These feature representations are concatenated and passed through a final MLP to predict the scalar value estimate.

The hyperparameters used in model components are summarized in Table 7.

B. Training Details

Data Processing. We start from long-horizon raw data collected continuously within a single scenario or location. Each raw sequence is segmented into a set of fixed-length episodes, each spanning 20 seconds. For each episode, we retain only the egocentric RGB images and the corresponding robot pose trajectories, discarding other sensor modalities. All data are temporally downsampled to 10 Hz to ensure a consistent representation across datasets and training stages.

For each episode, we construct a structured representation consisting of a history window, a current frame, and a future trajectory. The history contains past frames with relative pose offsets, while the future trajectory is represented as a sequence of relative motion offsets with respect to the current pose. These offsets are expressed in the robot-centric coordinate frame. Episodes are further subsampled at a fixed temporal stride to reduce redundancy, resulting in one annotated sample approximately every three seconds. This preprocessing yields a compact yet diverse set of training samples suitable for both

Table 7. Model Parameters

Parameter	Value
Token dim.	896
Proj. dim.	512
Dropout rate-KV cache	0.1
Dropout rate-state	0.5
Cross-attn layers	6
Query dim.	512
Queries (T)	30
FFN dim.	2048
Horizon (s)	3
Conv 1D layers	3

Table 8. Training Parameters

Parameter	Value
LR-Action	2×10^{-4}
LR-VLM	2×10^{-5}
Weight decay	1×10^{-2}
Batch size-VLM	2 per GPU
Batch size-Action	16 per GPU
Epoch	10
LR sched.	Cosine
Warmup	1000 step
Grad. clip	1.0

Table 9. RL Parameters

Parameter	Value
Action LR	1×10^{-5}
Value LR	1×10^{-4}
Rollout steps	1024
Gamma	0.99
GAE Lambda	0.95
Clip ratio	0.2
Grad. clip	0.5
Entropy coef.	0.001
PPO epochs	5
Batch size	64

vision-language modeling and control learning.

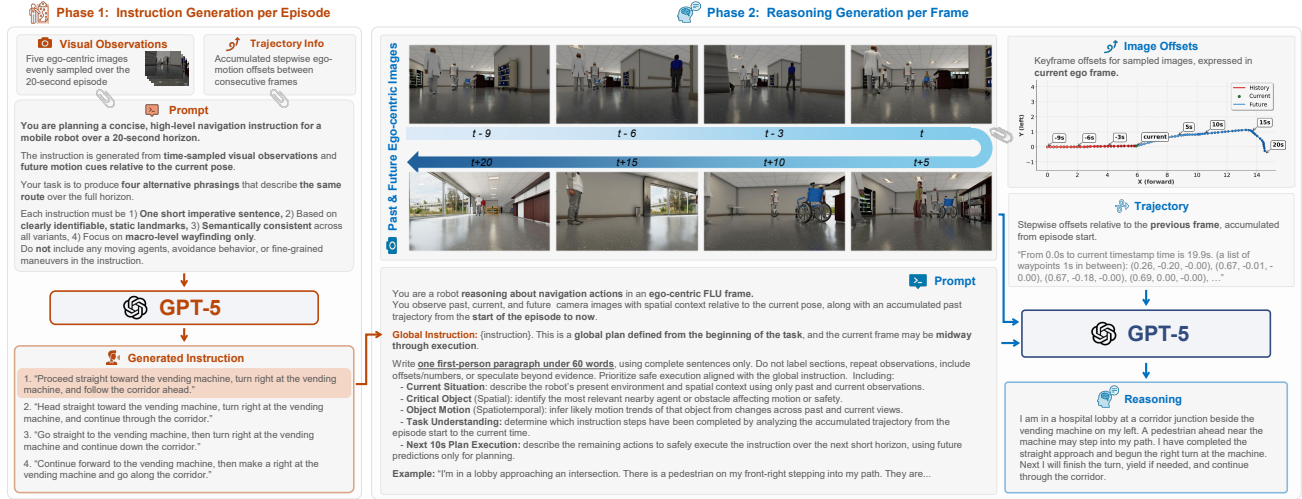


Figure 7. Overview of the annotation pipeline for VLM SFT. Representative frames and future trajectory information are used to generate long-horizon navigation instructions and concise CoT reasoning annotations.

VLM Supervised Fine-Tuning. We first perform supervised fine-tuning (SFT) of the VLM. For each 20-second episode, we automatically generate a long-term navigation instruction using GPT-5. To provide sufficient temporal context, we select five representative frames corresponding to the current time step and future time offsets at 5, 10, 15, and 20 seconds. These frames, together with the future trajectory offsets, are used to prompt the language model to generate four semantically equivalent instruction variants describing the same route. The variants differ only in wording and are randomly assigned across episodes to increase linguistic diversity while preserving route consistency. Figure 7 illustrates the annotation pipeline for navigation instruction generation and scenario reasoning on the DynaNav simulation data, while Figure 8 presents representative examples of instruction and reasoning annotations from the GND and SCAND datasets.

In addition to language instructions, we annotate each episode with a concise reasoning trace. The reasoning trace is generated by conditioning GPT-5 on a richer temporal context that includes the past trajectory, observations at 3, 6, and 9 seconds in the past, the current observation, and future observations. The past trajectory is summarized at 1s intervals from the start of the episode to the current time, expressed in the local frame of the previous 1s observation, to help the model infer motion patterns and task progress. The generated CoT focuses on five aspects: (1) the current situation, (2) critical objects, (3) object motion, (4) task progress relative to the instruction, and (5) the next short-horizon plan. To ensure efficiency and consistency, the reasoning trace is constrained to be brief and is stored alongside the episode.

During SFT, the vision encoder is frozen. The language model is trained to autoregressively generate either (1) a reasoning-augmented sequence followed by waypoint predictions or (2) waypoint-only outputs. Mixing these two target formats allows



Figure 8. Examples of navigation instruction and CoT reasoning annotations from the GND and SCAND datasets.

the VLM to flexibly produce explicit reasoning when required and output only waypoints to maintain efficiency.

Imitation Learning with Delayed Inference. After fine-tuning the VLM, we train the action expert via imitation learning while explicitly accounting for inference latency. To simulate real-world deployment conditions, we randomly sample inference delays and expose the action policy to temporally delayed VLM inference hidden states together with explicit latency metadata. During training, the policy is conditioned on delayed ground-truth reasoning annotations and guidance waypoints, ensuring access to the correct high-level intent. The overall training procedure is summarized in Algorithm 1.

Algorithm 1 Imitation Learning with Delayed Inference

Require: Demonstration dataset $\mathcal{D}$, action policy π_θ , frozen VLM f_{LM}

- 1: **for** each training iteration **do**
 - 2: Sample instruction, current visual observation, and expert future trajectory, $(I, x_t, \{\hat{\mathbf{p}}_t\}) \sim \mathcal{D}$
 - 3: Sample inference delay $\Delta t \sim U(0, 10)$
 - 4: Retrieve delayed visual observations and reasoning traces $(\mathcal{X}_{t-\Delta t}^{\text{vlm}}, \mathcal{R}_{t-\Delta t}) \sim \mathcal{D}$
 - 5: Obtain delayed VLM hidden state (KV cache) $S_{t-\Delta t} \leftarrow f_{\text{LM}}(\mathcal{V}_{t-\Delta t}^{\text{vlm}}, I, \mathcal{R}_{t-\Delta t})$
 - 6: Extract current visual tokens $\mathcal{V}_t^{\text{act}} = \text{Enc}_{\text{vis}}(x_t)$
 - 7: Encode robot state s_t and latency metadata $(\Delta t, \Delta x, \Delta y, \Delta \theta)$
 - 8: Predict action chunk $\{a_t^{(1:T)}\} \leftarrow \pi_\theta(\mathcal{V}_t^{\text{act}}, S_{t-\Delta t}, s_t, \Delta t, \Delta x, \Delta y, \Delta \theta)$
 - 9: Integrate the predicted actions to obtain trajectory $\{\mathbf{p}_t^{(1:T)}\}$
 - 10: Compute imitation loss $\mathcal{L}_a$
 - 11: Update θ by minimizing $\mathcal{L}_a$
 - 12: **end for**
-

For the waypoint-guided policy variant, we transform the delayed VLM-predicted waypoints into the coordinate frame of the robot’s current position before conditioning the action policy. The same transformation is applied consistently during both training and inference. The parameters used for training are provided in Table 8.

Online Reinforcement Learning. At each control step, the action expert predicts a short-horizon trajectory, from which we select a target corresponding to one second into the future. This target is converted into a desired linear velocity and angular velocity, which define the mean of a Gaussian action distribution. The policy additionally maintains a learnable log-standard-deviation parameter, yielding a stochastic Gaussian policy suitable for on-policy optimization. To preserve consistency with deployment conditions, the policy continues to operate on temporally delayed VLM inference hidden states and latency metadata during reinforcement learning. Delayed VLM hidden states are cached during rollout collection and reused during policy optimization. We adopt Proximal Policy Optimization (PPO) (Schulman et al., 2017) with a learned value function to optimize the policy under this delayed-inference setting. The overall procedure is summarized in Algorithm 2 and the parameters used for PPO are provided in Table 9.

Algorithm 2 Online Reinforcement Learning

Require: Action policy π_θ , value network V_ψ , frozen VLM f_{LM} , environment $\mathcal{E}$

- 1: **for** each PPO iteration **do**
- 2: Initialize rollout buffer $\mathcal{B} \leftarrow \emptyset$
- 3: **for** each environment step t **do**
- 4: Observe current visual observation x_t and robot state s_t
- 5: Run VLM inference asynchronously in the background
- 6: Retrieve delayed VLM inference hidden state $S_{t-\Delta t}$ and latency metadata $(\Delta t, \Delta x, \Delta y, \Delta \theta)$
- 7: Extract current visual tokens $\mathcal{V}_t^{\text{act}} = \text{Enc}_{\text{vis}}(x_t)$
- 8: Compute Gaussian policy parameters $(\mu_t, \log \sigma) \leftarrow \pi_\theta(\mathcal{V}_t^{\text{act}}, S_{t-\Delta t}, s_t, \Delta t, \Delta x, \Delta y, \Delta \theta)$
- 9: Sample action $a_t \sim \mathcal{N}(\mu_t, \sigma^2)$
- 10: Execute a_t in environment $\mathcal{E}$ and observe reward r_t and next state
- 11: Store data $(x_t, S_{t-\Delta t}, s_t, a_t, r_t, \Delta t, \Delta x, \Delta y, \Delta \theta)$ in $\mathcal{B}$
- 12: **end for**
- 13: Compute returns $\hat{R}_t$ and advantages $\hat{A}_t$ using value network V_ψ
- 14: **for** each PPO optimization epoch **do**
- 15: Update policy π_θ using clipped surrogate objective
- 16: Update value network V_ψ by minimizing value regression loss
- 17: **end for**
- 18: **end for**

C. Benchmark Details

We evaluate navigation methods using a language-conditioned physics-based benchmark consisting of 85 episodes across four environments: *Hospital*, *Office*, *Outdoor*, and *Warehouse*. Each episode specifies a unique combination of scene layout, start pose, goal location, and natural-language instruction. Task difficulty is systematically increased by varying pedestrian density (from 0 to 200 agents), instruction complexity, and time limits, while maintaining consistent goals within each task. Pedestrian agents exhibit diverse visual appearances and body types to increase visual variability and reduce reliance on appearance-specific cues. Figure 9 illustrates navigation instructions and scenarios included in the benchmark.

For each environment, we design multiple navigation tasks that share the same semantic objective but differ in crowd density, robot platform (Nova Carter or Spot), and timeout constraints. This design enables controlled evaluation of robustness to dynamic obstacles, long-horizon instruction following, and cross-robot generalization. An episode is considered successful if the robot reaches the target location within a predefined distance threshold before the time limit; outdoor environments use a relaxed threshold to account for large-scale spatial uncertainty. All experiments are conducted with a fixed random seed to ensure reproducibility.

Hospital Environment. We conducted 25 episodes in a large hospital scene featuring long corridors, patient rooms, medical equipment, and service areas. Tasks involve reaching vending machines, hospital beds, water dispensers, and hallway endpoints under dense pedestrian traffic. The hospital setting is characterized by narrow passages, frequent occlusions, and socially constrained motion, making it particularly challenging for safe navigation under crowd interference. For each task type, we systematically vary the number of simulated pedestrians from 0 to 60 and proportionally increase the episode timeout. This setting evaluates robustness to crowd density and social navigation challenges in structured indoor spaces.

Office Environment. We perform 25 episodes in a complex office environment composed of interconnected rooms, corridors, elevators, and desk areas. Tasks emphasize multi-step spatial instructions involving turns, room entries, and landmark-based goals (e.g., desks, carpets, doors), often beyond immediate perceptual range. The office environment introduces greater topological diversity and branching layouts, requiring precise instruction grounding rather than corridor following. Pedestrian density is again increased from 0 to 60 to study performance degradation under dynamic human participants in semantically rich but spatially compact environments.

Warehouse Environment. We evaluate 25 episodes in a structured warehouse scene with long aisles, racks, forklifts, cones, and fire extinguishers. The tasks emphasize fine-grained spatial reasoning, such as selecting the correct aisle, identifying object instances in repetitive layouts, and navigating to precise rack locations. The warehouse presents high visual repetition with minimal semantic diversity, stressing the model’s ability to disambiguate similar structures under motion and crowd

interference. Pedestrian density is varied from 0 to 60 with adjusted timeouts to control task difficulty. Owing to the warehouse’s compact spatial layout relative to the office and hospital environments, this setting represents the most crowded and visually ambiguous scenario.

Outdoor Environment. We include 10 episodes in a large-scale outdoor urban environment with uneven terrain, ramps, streets, and large open spaces. These tasks involve long-distance navigation guided by distant landmarks, such as buildings, umbrellas, fountains, and storefronts. The number of pedestrians ranges from 100 to 200, and a relaxed success threshold is used to account for localization uncertainty and large-scale goals. Due to the substantially higher rendering and simulation cost of large outdoor scenes, we limit this setting to 10 episodes while maintaining high environmental diversity. These episodes test scalability, visual clutter robustness, and instruction grounding in outdoor settings, representing the most demanding setting in terms of perception, planning horizon, and real-time control stability.

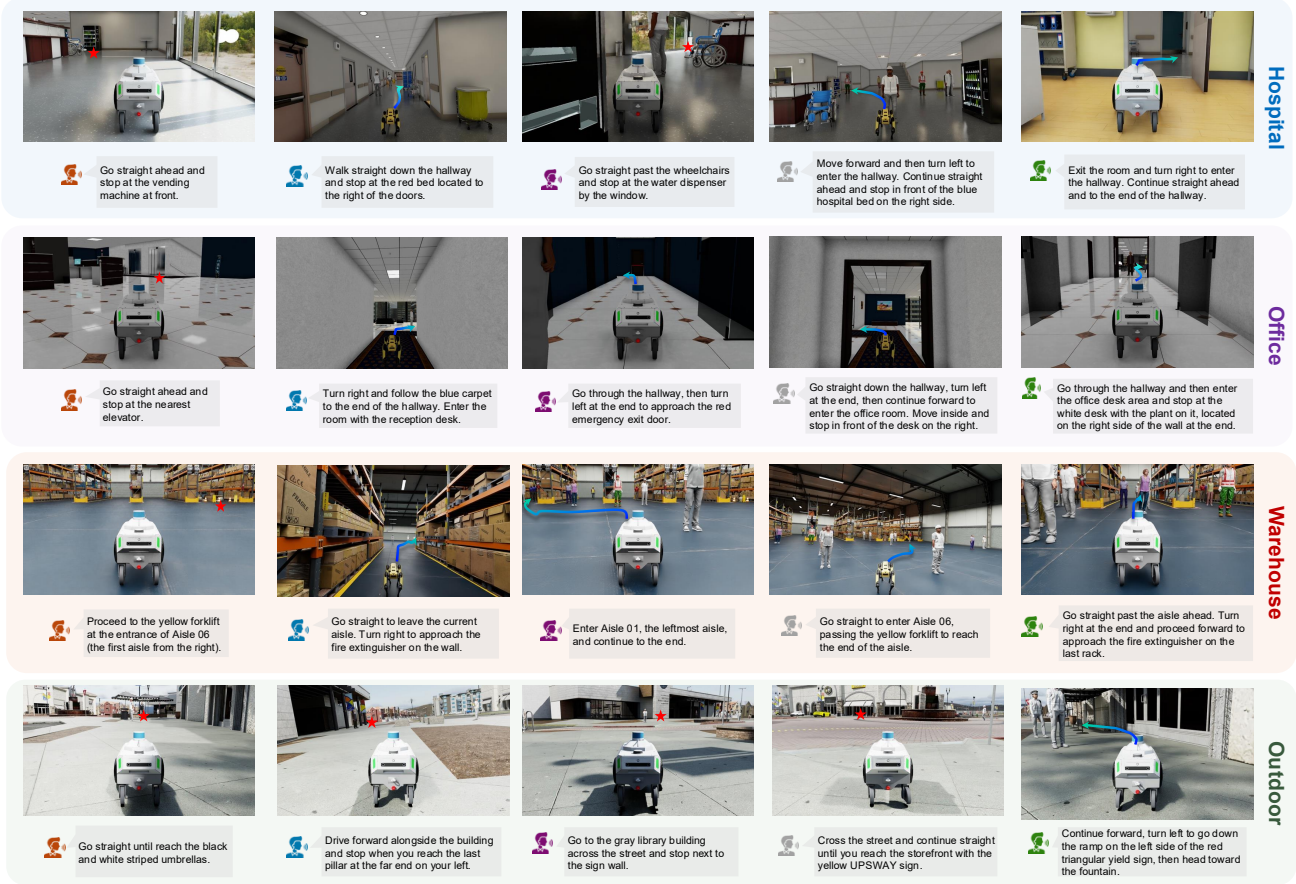


Figure 9. Overview of the DynaNav benchmark. Task instructions are provided, as well as corresponding navigation scenarios across the Hospital, Office, Outdoor, and Warehouse environments, highlighting variations in scene layout, landmarks, and human density.

D. Experiment Details

Data Collection. We collect demonstration data by asking a human expert to teleoperate the robot using a keyboard across four designed environments (warehouse, office, hospital, and outdoor) within the DynaNav simulation. Each environment contains dynamic human participants. The expert navigates the robot toward predefined goals, and upon completing each demonstration, records a corresponding natural language instruction describing the navigation task. The resulting dataset consists of RGB images and trajectories of robot camera poses. In total, we collected 310 episodes, corresponding to 5.1 hours of navigation data.

Online Reinforcement Learning. We perform online RL of the TIC-VLA model on three different tasks (different from testing) across three environments: office, hospital, and warehouse. The environment operates at a control frequency of 10 Hz, corresponding to an environment step of 0.1 seconds. During training, the VLM runs inference asynchronously in

the background upon receiving new observations and returns its outputs to the action policy once available. We finetune only the final cross-attention Transformer layer and the MLP decoder of the action expert. To emulate deployment on resource-constrained hardware, we explicitly inject random inference delays before releasing the VLM outputs to the policy, with an average delay of approximately 50 environment steps (5 seconds). The three tasks and environments, illustrated in Figure 10, are rotated every 100 PPO iterations.

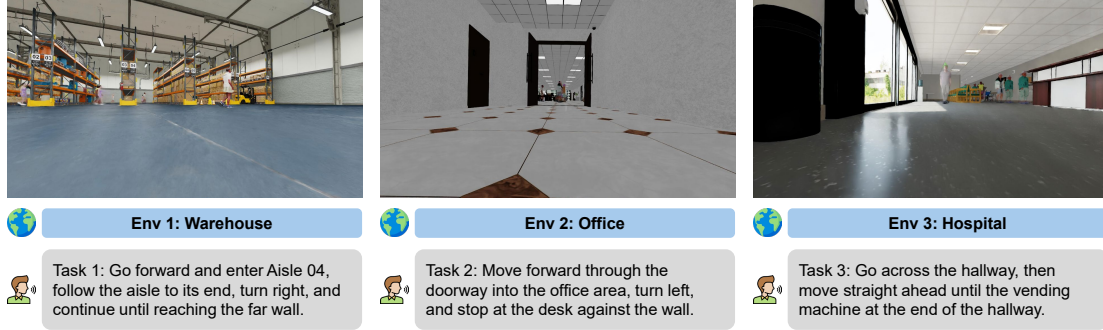


Figure 10. Online reinforcement learning tasks used to train TIC-VLA across three environments and tasks.

The weights for the reward function (Equation (5)) are set to $w_g = 400$ for reaching the goal, $w_p = 5$ for progress toward the goal, $w_c = -100$ for collisions with obstacles and humans, and $w_s = -0.1$ for the speed penalty. The target speed for the speed penalty is set to 1 m/s.

Baseline Methods. We compare our approach against several baseline methods. The behavior cloning (BC) baseline is trained directly on the demonstration dataset without modeling inference latency or delayed perception, while being provided with privileged access to the relative goal position. For the RL baseline, we train a PPO agent whose policy shares the same architecture as the value network, except that it directly outputs the action mean and standard deviation. The PPO policy is trained separately for each environment using 10 parallel environments and a total of 1M steps per environment. For other baselines, including Uni-NaVid, NAVILA, NavDP, DualVLN, MobileVLA, and OmniVLA, we follow their respective open-source implementations and training.

Ablation Action Policy Architectures. We implement the diffusion-based action policy as a conditional denoising model over fixed-length action chunks of $T = 30$ steps. The forward diffusion process uses $K = 100$ noise steps with a cosine variance schedule, and the denoiser is a Transformer backbone aligned with the query-based policy to ensure comparable capacity. Noisy action tokens are augmented with timestep embeddings and positional encodings, while conditioning information from the vision encoder and cached VLM hidden states is injected at every denoising layer via cross-attention. Actions are normalized prior to training, and the model is trained with the standard denoising mean-squared-error objective to predict clean actions. During inference, we apply DDIM sampling with 5 steps, which provides an efficient trade-off between computation and trajectory quality while preserving multimodal behavior.

The flow-based action policy models the conditional distribution of action chunks using a normalizing flow with a standard Gaussian base distribution, enabling exact likelihood optimization. The flow is composed of multiple coupling layers with learned affine transformations, and each layer is conditioned on the same vision features and cached VLM hidden states as the diffusion model to ensure architectural parity. Actions are standardized to zero mean and unit variance, and training maximizes the conditional log-likelihood of expert demonstrations. At inference, action sequences are generated by sampling from the Gaussian base and applying the inverse flow with 5 flow steps; reducing the number of steps leads to noticeably degraded performance.

Simulation Settings. All experiments are conducted in a high-fidelity Isaac Sim simulator running at 30 Hz. The navigation policy operates at a control frequency of 10 Hz; actions are applied once every three simulation frames and held constant in between. Each episode is executed in an isolated process with a fresh simulator instance, ensuring clean initialization and reproducibility. Episodes terminate when the robot reaches the goal (defined by a 2D distance threshold of 1.5 m) or when a timeout is reached. Robot trajectories are recorded at the simulation rate to compute path length and SPL. Collisions are detected both with animated human characters using distance proximity (threshold of 0.5 m) and with static scene objects using a contact sensor. All metrics are aggregated across episodes and reported using standard navigation benchmarks.

Testing Settings. We employ a closed-loop simulator running at 30 Hz, while executing the policy at 10 Hz (every 3

Algorithm 3 Closed-loop Rollout with Asynchronous VLM Reasoning

Require: Environment $\mathcal{E}$ (30 Hz), wrapper TIC-VLA, controller $\mathcal{C}$, instruction I , goal g

```

1:  $s_0 \leftarrow \mathcal{E}.\text{RESET}()$ 
2: image buffer  $\mathcal{B} \leftarrow \emptyset$ 
3: reset episode state()
4: first_kv_ready  $\leftarrow \text{False}$ 
5:  $(v_0, \omega_0) \leftarrow (0, 0)$  {initial controls}
6:  $(v_k, \omega_k) \leftarrow (v_0, \omega_0)$ 
7: for  $k = 0, 1, 2, \dots$  do
8:    $\text{pose}_k \leftarrow \mathcal{E}.\text{OBSERVEPOSE}()$ 
9:    $\text{img}_k \leftarrow \mathcal{E}.\text{RENDERCAMERA}()$ 
10:  append  $\text{img}_k$  to  $\mathcal{B}$ 
11:  if  $k \bmod 3 = 0$  then
12:     $(\mathcal{X}_k^{\text{act}}, \mathcal{X}_k^{\text{vlm}}) \leftarrow \text{SAMPLEFRAMESAVAILABLE}(\mathcal{B})$ 
13:     $s_k^{\text{robot}} \leftarrow [v_{x,k}, v_{y,k}, \omega_{z,k}]$  {robot state only; latency metadata is computed inside predict_async}
14:     $(r_k, \mathbf{w}_k, k_{\text{gen}}, \text{kv\_ready}, \text{pose}_{\text{gen}}) \leftarrow \text{predict\_async}(\mathcal{X}_k^{\text{act}}, \mathcal{X}_k^{\text{vlm}}, I, s_k^{\text{robot}}, k, \text{pose}_k)$ 
15:    if  $\neg \text{first\_kv\_ready} \wedge \text{kv\_ready}$  then
16:      first_kv_ready  $\leftarrow \text{True}$ 
17:    end if
18:    if  $\neg \text{first\_kv\_ready}$  then
19:       $(v_k^*, \omega_k^*) \leftarrow (0, 0)$ 
20:    else
21:       $(v_k^*, \omega_k^*) \leftarrow \mathcal{C}(\mathbf{w}_k)$ 
22:    end if
23:     $(v_k, \omega_k) \leftarrow \text{SMOOTH}(v_k^*, \omega_k^*)$ 
24:  end if
25:   $\mathcal{E}.\text{STEP}(v_k, \omega_k)$  {termination: success / timeout}
26: end for

```

simulation steps), as summarized in Algorithm 3. At each 10 Hz tick, the agent samples a short visual context $\mathcal{X}_k$ from the image buffer using `SAMPLEFRAMESAVAILABLE`, which selects all available frames among the target offsets, e.g., $\{t-9, t-6, t-3, t\}$, and falls back to fewer frames when the full history is not yet available. Latency-aware metadata $(\Delta t, \Delta x, \Delta y, \Delta \theta)$ is computed inside `predict_async` from the timestamp and pose associated with the cached VLM state. The controller $\mathcal{C}$ then consumes the policy output $\mathbf{w}_k$ to produce smoothed velocity commands (v_k, ω_k) , and the environment advances with $\mathcal{E}.\text{STEP}(v_k, \omega_k)$.

Asynchronous inference. VLM reasoning and KV-cache extraction are executed asynchronously via `predict_async` (Algorithm 4), while the action policy continues to run at 10 Hz. Each call first polls the background VLM job and, if it has completed, updates the cached response r^{cache} , cached KV features kv^{cache} , and the timestamp and pose associated with the cached semantic state. If no generation is currently running, a new background generation job is launched using the VLM visual context $\mathcal{X}_k^{\text{vlm}}$ and instruction I . While the new VLM job is in flight, the action policy reuses the most recent available KV cache features to decode $\mathbf{w}_k$ from the current observation, robot state, and latency metadata.

Synchronous inference. In the synchronous baseline, VLM inference is executed in a blocking manner, during which the robot halts and no control commands are issued. At each decision step, the robot remains stationary while the VLM processes the current visual context and instruction, ensuring that semantic reasoning is generated from up-to-date observations. As a result, the effective semantic delay satisfies $\Delta t = 0$, and the corresponding ego-motion offsets $(\Delta x, \Delta y, \Delta \theta)$ are all zero. Once inference completes, the robot executes the predicted trajectory for a fixed horizon of 0.5 s, after which it again halts for the next VLM inference cycle. This design eliminates semantic staleness but introduces intermittent control stalls due to blocking inference.

Inference without reasoning. For ablations that remove explicit VLM reasoning, we keep the same closed-loop rollout (Algorithm 3) and the same asynchronous interface (Algorithm 4), but disable VLM generation entirely. Concretely, the background VLM job performs a forward pass on the visual-language input $\mathcal{X}_k^{\text{vlm}}$ and returns only the extracted KV cache;

no tokens are decoded, and the textual response is not produced. As a result, r_k is always **None**, and control depends only on the cached KV signal and current observations.

Latency control in simulation. We evaluate robustness under effective semantic delay, defined as the elapsed time between observation capture and semantic state availability. To isolate the effect of delay, we keep the semantic model fixed and vary the scheduling of semantic updates. The default asynchronous setting yields an average effective latency of about 2 s with full semantic reasoning. For a lower-latency condition (1 s), we pause the simulation after 1 s of elapsed time and resume it once VLM inference completes, yielding a reduced effective delay. For higher-latency settings (3-5 s), visual observations are captured immediately, but VLM inference is launched after a delay of 30, 60, or 90 simulation frames (30 Hz). During this period, the policy continues executing with the most recent cached semantics, and newly generated responses and KV caches are applied immediately upon completion. This increases effective semantic delay without altering simulation dynamics.

Algorithm 4 predict_async (asynchronous VLM generation and action policy)

```

1: Function predict_async( $\mathcal{X}_k^{\text{act}}, \mathcal{X}_k^{\text{vlm}}, I, s_k^{\text{robot}}, k, \text{pose}_k$ )
2: /* persistent internal state stored across calls */
3:   gen_job {background VLM generation job, or None}
4:   kv_cache {most recent extracted KV cache}
5:   r_cache {most recent completed VLM response}
6:   k_cache, pose_cache {step and pose associated with the cached KV}
7:   POLLRUNNINGGENERATION( $k$ ) {if a background generation finishes, update kv_cache, r_cache, k_cache, pose_cache}
8:   k_gen  $\leftarrow$  None
9:   pose_gen  $\leftarrow$  None
10:  if gen_job is inactive then
11:    ( $k_{\text{gen}}, \text{pose}_{\text{gen}}$ )  $\leftarrow$  ( $k, \text{pose}_k$ ) {record when and where this generation starts}
12:    start background VLM generation using  $\mathcal{X}_k^{\text{vlm}}, I, k_{\text{gen}}$ , and  $\text{pose}_{\text{gen}}$ 
13:    gen_job becomes active
14:  end if
15:  if kv_cache is undefined then
16:    kv  $\leftarrow$  None
17:     $r_k \leftarrow r_{\text{cache}}$ 
18:    kv_ready  $\leftarrow$  False
19:    ( $\Delta t, \Delta x, \Delta y, \Delta \theta$ )  $\leftarrow$  (0, 0, 0, 0)
20:  else
21:    kv  $\leftarrow$  kv_cache
22:     $r_k \leftarrow r_{\text{cache}}$ 
23:    kv_ready  $\leftarrow$  True
24:     $\Delta t \leftarrow \text{ELAPSEDTIME}(k, k_{\text{cache}})$ 
25:    ( $\Delta x, \Delta y, \Delta \theta$ )  $\leftarrow$  RELATIVEPOSE( $\text{pose}_{\text{cache}}, \text{pose}_k$ )
26:  end if
27:   $\mathbf{w}_k \leftarrow \pi_{\theta}(\text{EMBED}(\mathcal{X}_k^{\text{act}}), s_k^{\text{robot}}, \text{kv}, \Delta t, \Delta x, \Delta y, \Delta \theta)$  {use current vision, robot state, cached semantic KV, and latency metadata}
28:  return ( $r_k, \mathbf{w}_k, k_{\text{gen}}, \text{kv\_ready}, \text{pose}_{\text{gen}}$ )
    
```

Evaluation Metrics. We evaluate navigation performance using standard goal-reaching metrics. Success (S) for a single episode is defined as whether the robot reaches the goal within a 2D Euclidean distance threshold $T_{\text{goal}} = 1.5\text{m}$:

$$S = \mathbb{I}(\|\mathbf{p}_{t,xy} - \mathbf{g}_{xy}\|_2 < T_{\text{goal}}), \quad (6)$$

where $\mathbf{p}_{t,xy}$ denotes the robot position at termination and $\mathbf{g}_{xy}$ denotes the goal position. Success Rate (SR) is reported as the average success over all testing episodes.

Navigation Error (NE) measures the final 2D distance to the goal:

$$\text{NE} = \|\mathbf{p}_{t,xy} - \mathbf{g}_{xy}\|_2. \quad (7)$$

Path Length (PL) is computed as the cumulative 2D distance traveled by the robot:

$$\text{PL} = \sum_{i=1}^{N-1} \|\mathbf{p}_{i+1,xy} - \mathbf{p}_{i,xy}\|_2, \quad (8)$$

where $\{\mathbf{p}_{i,xy}\}_{i=1}^N$ are robot positions.

Success weighted by Path Length (SPL) is defined as:

$$\text{SPL} = \text{SR} \cdot \frac{d_{\text{opt}}}{\max(d_{\text{opt}}, \text{PL})}, \quad (9)$$

where d_{opt} is the shortest distance from the start position $\mathbf{p}_{0,xy}$ to the goal.

Collision (C) is a binary episode-level indicator that equals 1 if any collision occurs during the episode and 0 otherwise:

$$\text{C} = \mathbb{I}(\exists t \text{ s.t. collision}_t), \quad (10)$$

Collisions include contacts with both animated human agents and static scene objects. The Collision Rate (CR) is computed as the fraction of test episodes in which at least one collision occurs.

Physical Robot Testing. We evaluate all methods on a Unitree Go2 quadruped robot. The primary computing setup consists of a laptop equipped with an NVIDIA RTX 4060 GPU and an NVIDIA Jetson Orin NX mounted onboard. For baseline methods whose memory or compute requirements exceed the capacity of these devices, we use a remote desktop equipped with an NVIDIA RTX A6000 GPU. In this setup, the remote machine performs model inference and communicates with the laptop that relays sensor observations and action commands to the robot.

E. Additional Results

Simulation Testing. We provide additional simulation testing results of TIC-VLA in diverse DynaNav environments, including outdoor, hospital, and warehouse, using multiple robot platforms, in Figure 11. The results demonstrate that TIC-VLA is able to generate coherent semantic reasoning under delayed perception and translate it into stable, low-level control commands. The robot consistently adapts its behavior to dynamic agents by yielding, adjusting speed, and selecting appropriate turning maneuvers, while following the given natural language instructions. To further illustrate temporal consistency and closed-loop behavior, dynamic video results are provided.

Real-world Testing. We report task-specific results from real-world robot navigation experiments. Each task is executed five times, and the average success rate (SR) is summarized in Table 10. As shown, TIC-VLA consistently outperforms prior methods across both indoor and outdoor environments. Figure 12 presents three representative real-world navigation examples under diverse conditions. To further demonstrate the closed-loop control behavior, we provide additional dynamic video results on the [project website](#).

Table 10. Real-world testing task-specific evaluation results.

Task	TIC-VLA SR (%)	DualVLN SR (%)	NaVILA SR (%)
Indoor Hallway	80	40	20
Indoor Office	80	20	0
Outdoor Campus	80	60	60
Outdoor Sidewalk	100	80	60
Average	85	50	35

Influence of Action Policy. Table 11 compares three action policy designs in TIC-VLA: diffusion-based, flow-based, and query-based Transformer policies. Overall, we observe no performance benefit from the more complex diffusion and flow-matching policies over a simple query-based Transformer. The query-based policy consistently achieves the best results across all metrics. By directly predicting action sequences in a single forward pass, it minimizes inference latency and preserves tight alignment between perception and control. Moreover, the simpler architecture facilitates more stable and efficient RL fine-tuning.

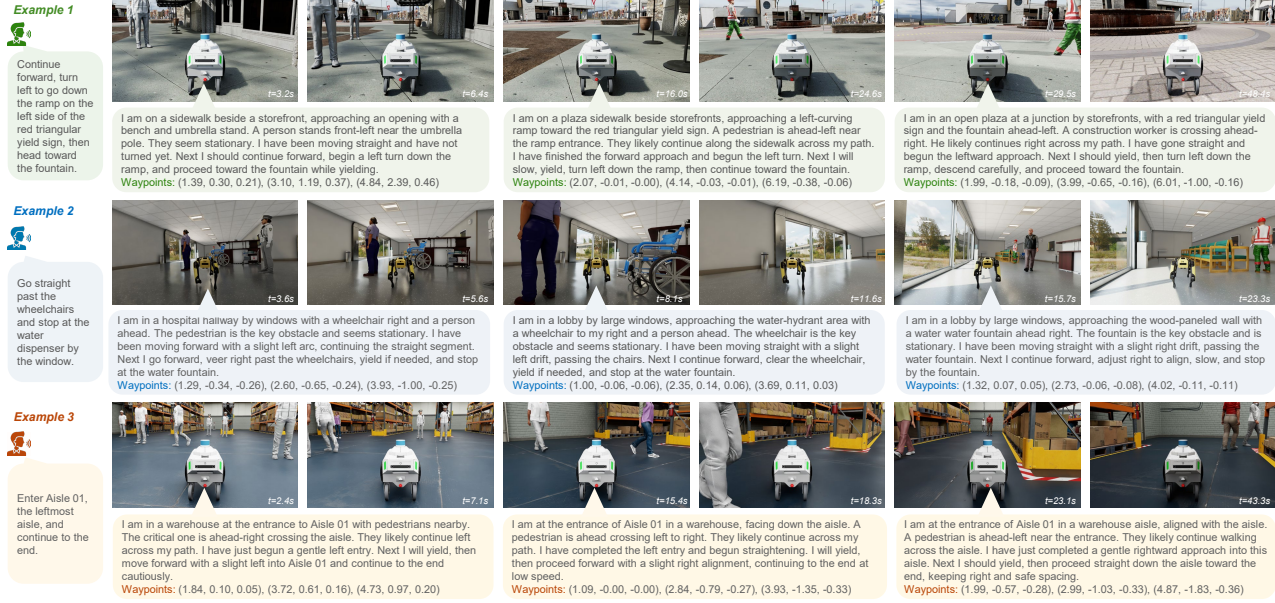


Table 11. Comparison of different action policies in the TIC-VLA model.

Policy Type	NE ($\downarrow$)	SR ($\uparrow$)	SPL ($\uparrow$)	CR ($\downarrow$)
Diffusion-based	14.30	38.82	35.41	41.18
Flow-based	15.04	32.94	29.65	45.88
Query-based (Ours)	10.85	47.06	42.41	34.12

Influence of VLM Backbone. We study the effect of VLM backbone scale and visual tokenization in TIC-VLA by comparing InternVL3 (1B) with SmolVLM2 (Shukor et al., 2025) (500M) and Qwen2.5-VL (Bai et al., 2025) (3B), as summarized in Table 12. The smaller SmolVLM2 benefits from lower inference latency but exhibits higher navigation error and lower success and SPL due to limited representational capacity. In contrast, the larger Qwen2.5-VL provides stronger visual understanding but incurs substantial inference delay, which degrades perception-action alignment and overall navigation performance. InternVL3 achieves the best trade-off between capacity and efficiency.

Table 12. Comparison of VLM backbone in the TIC-VLA model.

Backbone	NE ($\downarrow$)	SR ($\uparrow$)	SPL ($\uparrow$)	CR ($\downarrow$)
SmolVLM2 (500M)	14.82	38.82	31.06	41.18
InternVL3 (1B)	10.85	47.06	42.41	34.12
Qwen2.5-VL (3B)	13.26	32.94	29.98	35.29

Influence of Action Expert Architecture. We evaluate the sensitivity of TIC-VLA to the depth of the Transformer action expert by varying the number of cross-attention layers. All variants are evaluated without RL fine-tuning. As shown in Table 13, the 6-layer design achieves the strongest overall performance. The 3-layer variant performs comparably but is slightly weaker across most metrics, suggesting that a shallower action expert has limited capacity to integrate visual tokens, semantic features, and latency metadata. Increasing the depth to 12 layers does not improve performance and leads to a worse success rate, SPL, and collision rate despite a slightly lower navigation error. We therefore adopt the 6-layer action expert as the default architecture.

Sensitivity to Odometry Drift. We further evaluate robustness by injecting Gaussian noise into translation and rotation measurements during simulation. The noise magnitude is proportional to the traveled distance at each step and accumulates

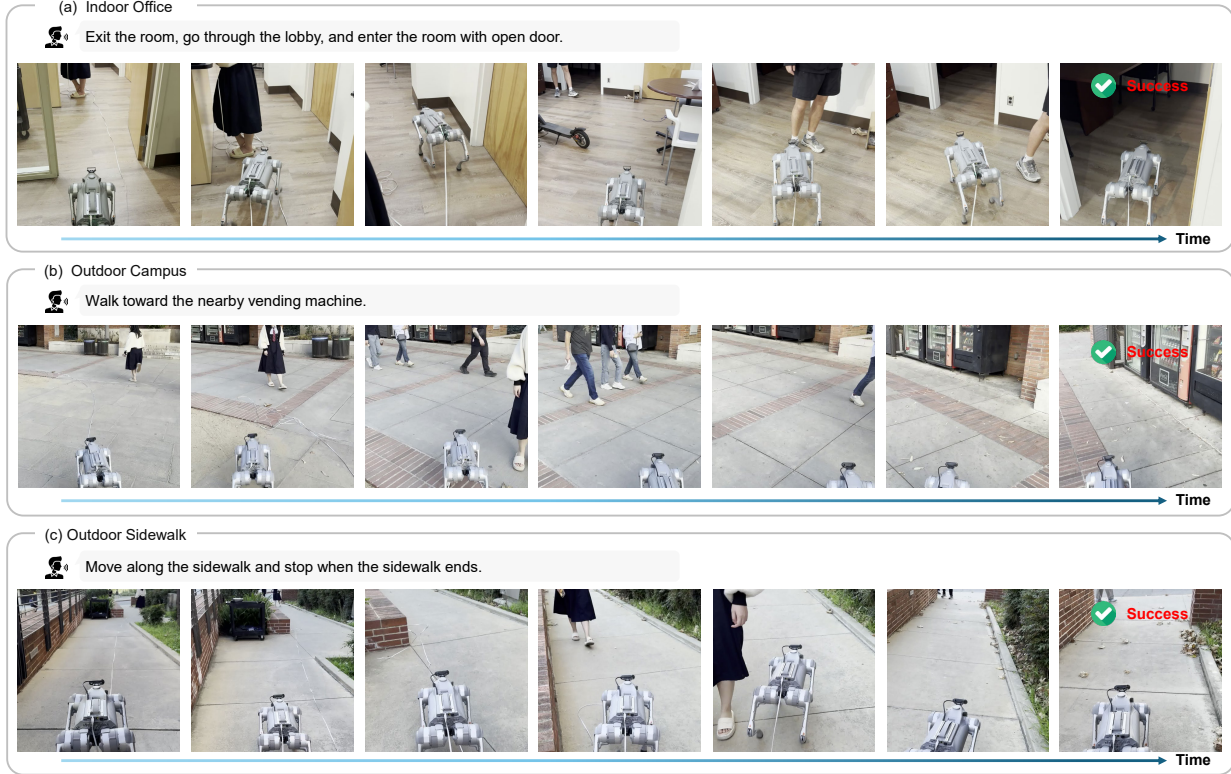


Figure 12. Real-world navigation examples demonstrating **TIC-VLA** executing language-conditioned navigation under real-time control.

Table 13. Effect of the number of cross-attention layers in the Transformer action expert.

Layers	NE ($\downarrow$)	SR ($\uparrow$)	SPL ($\uparrow$)	CR ($\downarrow$)
3	10.98	45.88	41.39	35.29
6 (ours)	10.85	47.06	42.41	34.12
12	10.73	44.71	40.77	37.64

over time, simulating realistic odometry drift. As shown in Table 14, TIC-VLA remains stable under typical drift conditions, with only marginal changes in navigation error, success rate, and SPL. Even under the severe stress-test setting, the policy continues to navigate successfully in a substantial portion of episodes, although performance degradation becomes more apparent due to accumulated localization errors. These results suggest that TIC-VLA is reasonably robust to moderate odometry inaccuracies and does not catastrophically fail under noisy motion estimation.

Table 14. Robustness of TIC-VLA under accumulated localization drift with Gaussian translation and rotation noise.

Drift Level	Gaussian Noise (trans, rot)	NE ($\downarrow$)	SR ($\uparrow$)	SPL ($\uparrow$)	CR ($\downarrow$)
None	—	10.55	55.29	50.29	28.24
Typical	(0.02 m/m, 1°/m)	10.66	55.29	50.12	29.41
Severe	(0.05 m/m, 5°/m)	11.87	43.53	38.41	35.29